

基于固定时域与预报融合的微网跨日随机调度

摘 要

微网需要在预测误差下提前购电，并用有限储能响应实时供需变化。库存的经济价值跨越午夜，合同又须先于当前净需求确定。针对滚动前瞻长度随日内时刻收缩、预报更新与购电承诺相互耦合的问题，本文建立统一凸能量模型，构造“固定 72 小时规划—成熟残差场景—因果合同—凸库存价值反馈”的闭环调度框架。

针对问题一，由合同计费凸性及同时充放电消元，建立确定性线性规划。典型日最优购电量为 **59,482.70 kWh**，费用为 **35,126.95 元**，日初和日末库存均为 6,000 kWh；独立原始—对偶核验支持该模型内最优值。

针对问题二，以固定 72 小时视域统一各次重规划的跨日前瞻，从近期成熟历史块中选取 7 条等权轨迹，结合因果仿射合同与三状态 Markov 库存价值反馈。2025 年 2 月至 12 月 334 日费用为 **1,381.79 万元**，较上一版同任务跨日策略节省 **2.62 万元**。完整轨迹重算表明，紧急购电费下降 **2.65 万元**，是净节省的主要来源。

针对问题三，将已发布官方光伏预报与历史预测按 **0.75 和 0.25** 融合，在合法发布节点修订剩余合同。远端年度复核汇总费用为 **1,335.24 万元**，较上一版同任务结果节省 **10.94 万元**。

针对问题四，按同一历史块联合构造净需求和价格场景，并在电池动作后按实际价格结算。远端汇总的日前、日内机制费用分别为 **1,459.24 万元、1,411.73 万元**，对应节省 **6.22 万元、13.48 万元**。四种机制采用同一组配置，比较统一真实初末库存 6,000 kWh。

理论上，本文给出单步与终点可达域、Bellman 凸性保持及斜率合并证明，使合同优化、连续库存动作和跨日执行形成一致的控制链。结果定位于**同年度候选比较**：问题二由完整轨迹验证，其余三项据远端汇总报告；费用差反映当前年度表现，适用范围由信息权限、物理条件和证据口径共同确定。

关键词：微网调度；固定滚动时域；预报融合；因果仿射策略；凸动态规划

1 问题重述与控制思路

微网购电有三种相互衔接的时间尺度：日前确定交付合同，日内按已发布预报修订承诺，电池每十分钟响应真实净需求。光伏富余时保留的电量可以服务当天晚峰，也可以留给次日；因此，电池库存应沿时间轴连续传递，并在真正评价终点统一核算。

四问共享母线平衡、储能约束和结算规则，依次扩展信息与交易权限。问题一在完整已知的典型日上安排购电与电池动作；问题二以历史信息确定不可日内调整的合同；问题三加入 0、6、12、18 时发布的光伏预报及合同调整；问题四进一步引入随机交付价格，分别考察日前和日内两种机制。本文将它们作为同一随机控制系统的特例。

本文的组织主线是让跨日视野稳定、让预报进入承诺、让库存价值进入实时动作。固定 72 小时视域使 0、6、12、18 时的重规划具有一致前瞻长度。合法预报与历史预测的融合提供当前净需求中心，成熟残差表达后续不确定性；上层优化符合信息截止时刻的合同，下层根据当前缺口和未来库存价值选择充放电。真实年末采用可达区间保证库存回到 6,000 kWh。

微网模型预测控制和多阶段储能规划提供了上述分层控制的研究基础 [1, 2, 3]。本文采用有限维因果扰动反馈表示合同策略 [4, 5]，以经验场景平均构造规划目标 [6, 7]，利用一维库存值函数的凸结构实现反馈。正文先解释物理机制，再给出模型和求解；实验分别回答年度账单、费用来源和时间稳定性三个问题。理论标记 A 表示数学性质，B 表示所声明模型内的最优性；其余实测结论按对应结果来源说明。

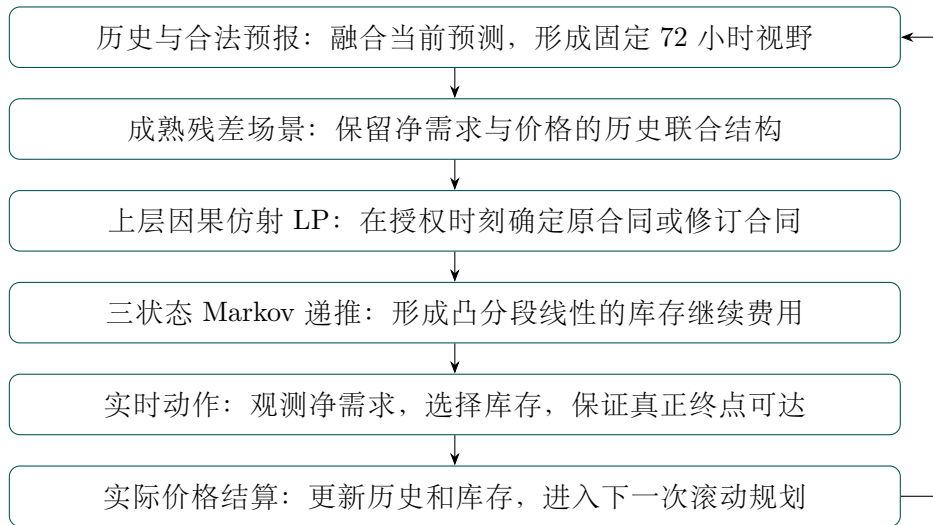


图 1 同一信息时序下的预测、合同规划与库存价值反馈

2 假设、符号与信息结构

2.1 模型假设

1. 附件功率按对应十分钟区间均值积分；电量统一在交流母线侧计量，内部库存单独递推。
2. 将题给 90% 解释为充电和放电各自的转换效率；不额外引入题面未给的自放电、线路损耗和循环退化参数。

3. 不向外网售电，允许免费弃置母线富余；紧急购电无题外容量上限。
4. 采用退购退款后支付 50% 违约金的主口径。
5. 合同只使用签约前已知信息；电池可观测本时段净需求，实际浮动电价在动作后用于结算。
6. 问题一选定周期初值 6000 kWh；问题二至四正式比较统一真实初末库存 6000 kWh。

2.2 物理参数、时间计量与符号

容量、功率与库存上下界取自题目。将每十分钟作为一个交付时段，时段长度为六分之一小时。输入的 00:10 至 24:00 数值解释为此前十分钟的区间均值，输出从 00:00–00:10 标记至 23:50–24:00。负荷与光伏功率除以 6 得到时段电量。单向效率 0.9 采用上文对题给 90% 的解释。于是

$$\eta = 0.9, \quad M = 5000/6, \quad E_{\min} = 1200, \quad E_{\max} = 10800. \quad (1)$$

额定充放电功率为 5,000 kW，允许库存为 1,200 至 10,800 kWh。系统允许以规定价格从外网补电，无额外外网容量限制；不向外网售电，母线富余可免费弃置且不设弃电上限。当前模型无循环奖励、启停时间和电池退化费用，价格非负。这些条件对应给定调度模型，也是后文消元及可达性命题的条件。弃电变量表示母线总富余，不能直接换算为纯光伏弃电率。合同必须先定，电池动作可以后做，真实电价只用于结算。

表 1 主要符号及计量口径

符号	含义
E_t, δ_t	时段初库存及本段库存增量, kWh
q_t, r_t	原始合同及最终有效交付合同, kWh
c_t, d_t, e_t, w_t	充电、放电、紧急购电、母线弃电, kWh
n_t, p_t	净需求电量及实际交付价格, kWh、元/kWh
ξ_t	外生扰动, 包括净需求、价格误差及预报创新
a, H, S	当前规划时段、该次时域长度、经验轨迹数
$K_t, V_{t,k}$	净需求箱标签、该标签下的库存继续费用函数
$\mathcal{F}_t^-, \mathcal{F}_t$	合同可用信息及当前净需求实现后的控制信息

2.3 严格的观测与行动次序

先签约，再观察当前净需求并操作电池，最后按实际交付价格结算。

$$\mathcal{F}_t^- \subseteq \mathcal{F}_t, \quad (q_t, r_t) \in L^0(\mathcal{F}_t^-), \quad (c_t, d_t, e_t, w_t) \in L^0(\mathcal{F}_t). \quad (2)$$

其中，原始合同还须对所属日签约时刻的信息可测，修订合同须对最后合法修订节点的信息可测；式(2)是共同必要条件，并不赋予逐时重新签约权。净需求实现后新增的观测不能倒流进入该时段合同。变动价机制的当段真实电价不作为动作输入，控制使用此前可得的预测价格，真实价格留在结算环节。

库存是物理状态，但一般不是完整随机控制状态。一个充分的状态表述须包含库存、未交付合同和足以更新条件分布的历史或预测器状态；也可以直接以完整合法历史定义策略，不预设有限维马尔可夫性。后文的二元状态是冻结本次名义合同后的经验近似状态，不是对原始随机过程充分状态性的证明。

3 统一凸能量模型

在题目的计费规则和电池约束下，合同费用是凸的；电价不为负时，存在不同时充放电的最优解，因而不必给电池加 0-1 开关。紧急购电关于库存变化同样是凸分段线性。若年末要回到 6,000 kWh，每一步就只能走到还来得及抵达终点的区间里。四问的差别不在这些等式，而在合同权限和信息集。

3.1 状态方程和随机控制目标

令净需求等于负荷电量减光伏电量，则电池递推与母线平衡为

$$E_{t+1} = E_t + \eta c_t - d_t / \eta, \quad (3)$$

$$r_t + e_t + d_t - c_t - w_t = n_t. \quad (4)$$

同时满足非负电量、容量边界及单段充放电量不超过 M 。对遵守各机制合同权限、信息约束和真正初末条件的策略集合，统一目标为

$$J^* = \inf_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=0}^{T-1} p_t \{ \phi(q_t, r_t) + 5e_t \} \right]. \quad (5)$$

这里的期望对应声明的外生过程分布。实际计算以有限经验场景上的样本平均近似该目标 [6, 7]，并作滚动执行。问题一为确定性且无紧急购电的特例；问题二固定价格并约束最终合同等于原合同；问题三放开合法修订并增加已发布预报；问题四使用随机价格结算。

3.2 合同结算凸性

采用题解中已经实现的退购解释：退购部分原电费由 50% 违约费替代，因此相对日初合同退回 50% 的减少部分电费。对每个交付段，计费相对原合同与最终合同净额计算，不把多次发布的变化绝对值重复累加。当最终合同增大时，费用对应原合同加 1.5 倍增购；减小时，原合同扣除 0.5 倍退购。因而

$$\begin{aligned} \phi(q, r) &= q + 1.5(r - q)^+ - 0.5(q - r)^+ \\ &= \begin{cases} 1.5r - 0.5q, & r \geq q, \\ 0.5r + 0.5q, & r < q, \end{cases} \\ &= \max\{1.5r - 0.5q, 0.5r + 0.5q\} = r + 0.5|r - q|. \end{aligned} \quad (6)$$

命题 1 (A：合同费用的凸性) 函数关于两个合同变量联合凸且分段线性。

证明. 两仿射表达式之差为最终合同减原合同，故在各自区间恰由较大者取值；仿射函数的逐点最大值为凸分段线性函数。□

因此可用上图变量 F 及两条线性不等式表示合同费用。非负价格保证最小化上图费用与原目标的最优值一致；价格为零时不要求所有上图变量均紧。合同修订没有引入新的非凸项，且与库存价值的凸结构兼容。若采用不同的退购结算解释，则须改变目标并重新计算，主结果均对应式(6)。

3.3 同时充放电消元与紧急电量

命题 2 (A: 存在单向最优解) 在上述免费无上限弃电、非负价格、无循环奖励条件下, 若原问题最优解存在, 则存在逐时不同时充放电的最优解。

证明. 对任一同时充放电时段, 构造

$$u = \min\{c, d/\eta^2\}, \quad c' = c - u, \quad d' = d - \eta^2 u. \quad (7)$$

二者保持非负且不超过原值, 至少一者为零。进一步有

$$\eta c' - d'/\eta = \eta c - d/\eta, \quad (c' - d') - (c - d) = -(1 - \eta^2)u. \quad (8)$$

令弃电增加同样的母线富余量, 保持合同和紧急电量不变, 即可保持平衡、库存与成本。逐时施加该变换得到单向最优解。该证明是存在性结论, 并不要求所有最优解均单向。□

以库存增量代替两个动作变量, 单向恢复公式为

$$\delta = E_{t+1} - E_t, \quad c = \delta^+/\eta, \quad d = \eta(-\delta)^+, \quad -M/\eta \leq \delta \leq \eta M. \quad (9)$$

其母线净耗电为两个仿射式的最大值, 最小紧急电量及相应弃电为

$$\begin{aligned} g(\delta) &= \max\{\eta\delta, \delta/\eta\}, \\ e(r, \delta; n) &= \max\{0, n - r + \eta\delta, n - r + \delta/\eta\}, \\ w &= r + e - g(\delta) - n. \end{aligned} \quad (10)$$

命题 3 (A: 即时费用的凸性) 对固定净需求、合同及非负价格, 紧急费用关于库存增量为凸分段线性函数。

证明. 式(10)是三个仿射函数的最大值, 乘以非负系数保持凸性及分段线性。□

这一消元把功率方向与供需追索压缩进一个库存变量。模型采用母线聚合平衡, 没有增加紧急电不得流向电池等分源约束; 相关结论均基于这一实际实现。

3.4 一步可达域与真正终点

命题 4 (A: 单步可达闭区间) 给定当前库存, 下一库存的物理可达域为

$$\mathcal{X}(E) = [E_{\min}, E_{\max}] \cap [E - M/\eta, E + \eta M]. \quad (11)$$

证明. 式(9)的增量上下界分别来自最大放电和充电; 加入容量界即得必要条件。区间中任一点均可由式(9)恢复合规单向动作, 并由紧急购电和免费弃电完成母线平衡, 故也充分。□

图中斜带随当前库存平移，对角线表示不动作，上下斜边表示最大充放电，容量边界在两端截断可行带。

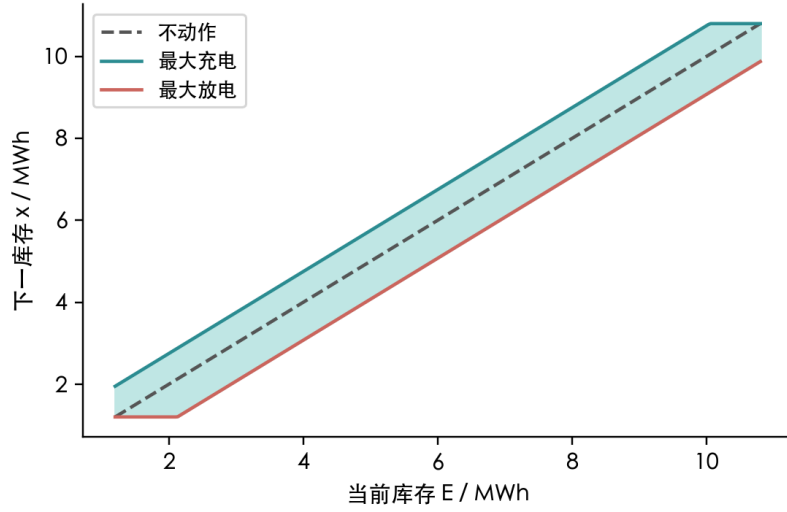


图 2 容量与功率共同决定的单步库存可达域

命题 5 (A: 真正终点的可达性) 本次动作后若还剩 N 步，终点目标位于容量区间，则下一库存必须且只须位于下列剩余可达区间；实际执行再与单步可达域相交：

$$\mathcal{R}_N = [E_{\min}, E_{\max}] \cap [E_T^* - N\eta M, E_T^* + NM/\eta], \quad E_{t+1} \in \mathcal{X}(E_t) \cap \mathcal{R}_N. \quad (12)$$

证明. 必要性来自剩余 N 步累计充、放电能力。充分性方面，当 N 为正且下一库存低于目标时，将差值均匀分配到剩余 N 次充电；高于目标时均匀分配到放电。区间条件保证每步不越功率界，单调趋近目标保证中间库存不越容量界。外网追索与弃电保证供需可行。 N 为零时区间退化为目标单点。□

本文主口径的真实评价终点为 12 月 31 日 24 时，目标 6,000 kWh。除每日闭合对照外，中间午夜不是终点；滚动规划域的末点也不自动施加真实终点等式。执行器把目标库存投影到式(12)的交集后恢复动作，从而保证真实末端可达。

4 确定性问题一：凸模型的基准验证

题目要求日初与日末库存相同，并未对该典型日另行固定周期库存。本文参照题给初始状态，将两端均设为 6000 kWh，并在这一明确条件下给出最优值。

问题一没有预测，也没有跨日：典型日的负荷、光伏和分时电价全部已知，当天首末库存均为 6,000 kWh，并禁止紧急购电。它用来核对凸模型能否给出题目要求的指定时段结果，也给后面的随机机制提供一个可对照的确定性基准。

典型日共 144 个交付段，供需和分时电价已知，原合同与最终合同相等，并禁止紧急电。模型可直接写为

$$\begin{aligned}
\min \quad & \sum_{t=0}^{143} p_t q_t, \\
& q_t + d_t - c_t - w_t = n_t, \\
& E_{t+1} = E_t + \eta c_t - d_t / \eta, \\
& 0 \leq c_t, d_t \leq M, \quad q_t, w_t \geq 0, \\
& E_{\min} \leq E_t \leq E_{\max}, \quad E_0 = E_{144} = 6000.
\end{aligned} \tag{13}$$

命题 2 允许保留连续松弛并恢复单向动作。所有约束线性，目标线性，使用 HiGHS 求解 [13]。**模型内结论：**求解器返回最优状态；独立线性规划与对偶账单在数值容差内相符。**结果：**全天购电 59,482.70 kWh，充电 20,740.67 kWh，放电 16,799.94 kWh，费用 35,126.95 元；原始对偶差约为 7.28×10^{-12} 元。该数值证书对应本确定性模型和浮点精度。

表 2 问题一购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量		59,482.70	全天购电费		35,126.95

表 3 问题一充放电与首末库存（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4,500.00	0.00	04:00–08:00	833.33	6,365.84
08:00–12:00	4,787.96	1,703.00	12:00–16:00	5,286.04	91.10
16:00–20:00	0.00	5,780.13	20:00–24:00	5,333.33	2,859.87
0:00 库存		6,000.00	24:00 库存		6,000.00

全天首末库存相同，故由状态递推求和，总放电量等于总充电量的效率平方倍，即 81%。图示库存随电价和供需共同变化：低价时段补能，高价值用电时段放能，最终回到规定库存。费用目标与外购电量目标不同，效率损耗并不妨碍低价向高价转移形成费用收益。

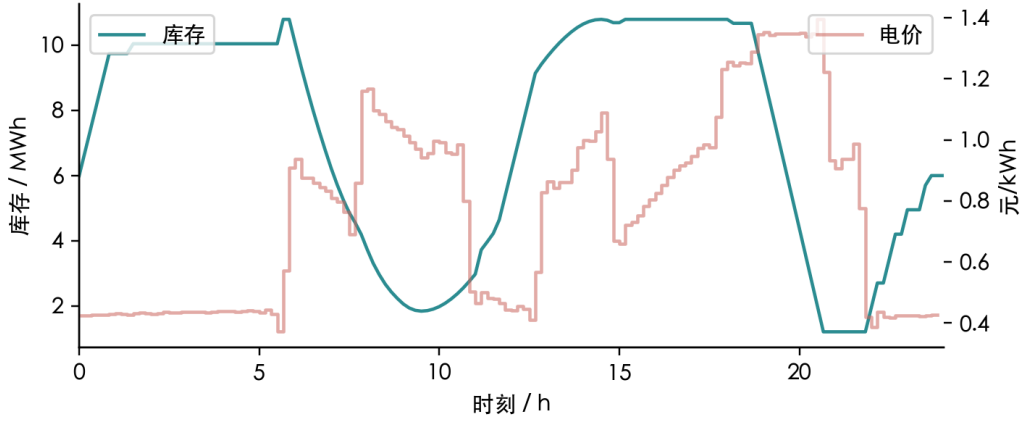


图 3 典型日库存与分时电价：解释跨时段移能机制

5 跨日随机规划：在合法信息中安排承诺

问题二至四的合同必须在真实净需求揭晓之前确定。规划只用当时已经允许看到的历史和预报，把已经走完的残差接到当前预测后面，作为若干条可能的后续轨迹；文件里尚未发生的数不得进入场景。电池如何响应当前缺口，留到下一节。

5.1 固定 72 小时：稳定日内各节点的跨日前瞻

设真正评价终点的绝对时段为 T_* ，每小时含 6 段。根节点 a 的核心末点和长度定义为

$$b_a = \min\{a + 432, T_*\}, \quad H_a = b_a - a. \quad (14)$$

在年末截断之前，四个日内重规划节点均前瞻 72 小时。上一版以第二个午夜为末点，日内四节点的实际长度分别为 48、42、36、30 小时。图4说明本次修改的直接作用：晚间重规划仍保留完整跨日视野，能够共同评估后续光伏、负荷和低价补能机会。72 小

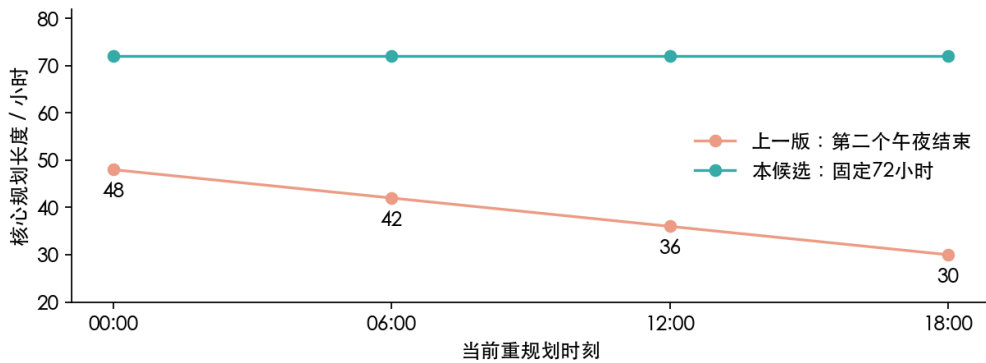


图 4 各次重规划的核心长度；真正年末均按剩余时长截断

时是滚动控制的配置参数。更长核心区间同时改变参与优化的交付段、可成熟历史块与末值价格窗口，因而其账单作用通过闭环比较评价；可行域包含关系不直接给出不同滚动策略的费用排序。

在 6、12、18 时启动的 72 小时规划于相同日内相位结束。此时完整继续状态还包含边界后尚未交付的原合同，代码采用库存线性末值近似而未求解这些合同的完整后续费用。核心区间外可用根节点预测作因果合同延拓，但该后缀不属于核心 LP 已优化的交付费用。实际运行在 0 时签订完整当日合同，每段都按原合同及最终合同结算，实际年度账单覆盖全部交付段。这一区分使有限时域的价值近似与真实交易执行保持清楚。

5.2 成熟残差与等权经验场景

预测族在 1 月 15 日至 31 日按 MAE 比较前一日、前一周、近七日均值和趋势四种候选；负荷选前一周、光伏选七日均值、价格选前一周。日内利用此前已观测的近 36 段误差作衰减修正。设当前绝对时段为 a ，规划长度为 H ，历史块起点为 h ，目标采用半开区间，则成熟条件为

$$[h, h + H) \subseteq [0, a), \quad h + H \leq a. \quad (15)$$

命题 6 (A: 成熟场景的非预见性) 若历史块满足式(15)，且预测器参数、残差重建和轨迹选取均由当前已知信息确定，则当前场景不依赖当前之后的真实目标。

证明. 历史块中的最后一个目标发生于 a 之前，所有真实目标均属于当前信息。历史原点预测的输入前缀按该原点截断；用当前已确定的预测族进行重建及残差运算，仍是当前已知数据的可测函数。按时间下标选取轨迹也不引入未来目标。只要求历史起点早于当前时点不足以保证这一结论。 $\square$

历史残差的重建使用冻结预测族，并非声称这些预测族在每个更早历史原点已经被实际选中。1 月开发段与预测族选择区间重叠，因而该段是内部开发比较；正式回放使用 2 月至 12 月。当前统一候选在年度比较中的定位见实验设计。

实际场景来自最近最多 28 个完整成熟残差块，按下标等距覆盖选取最多 7 条，赋予等权。净需求场景构造为当前预测加历史残差，价格场景使用同一历史块的价格残差：

$$n_{a+j}^s = \hat{n}_{a+j|a} + n_{h_s+j} - \hat{n}_{h_s+j|h_s}, \quad p_{a+j}^s = \max\{10^{-3}, \hat{p}_{a+j|a} + p_{h_s+j} - \hat{p}_{h_s+j|h_s}\}. \quad (16)$$

变动价格预测作正值截断；固定价机制直接使用给定分时价。此设计保留历史块内的联合时间结构，属于确定性时间覆盖的等权经验分布。其依据是保留已观测残差的块内形态；场景质量应按实际决策目标评价 [8]。场景是否适应当前规划，最终由同终点闭环账单判断。

5.3 有限维因果扰动反馈

合同与库存请求写成对已实现扰动特征的仿射反馈。仿射扰动反馈把策略搜索变成凸规划，是可调鲁棒优化与约束鲁棒控制中的标准参数化 [4, 5]；本文把它用在经验场景的期望费用上，并按题目的签约时刻截断特征，而不是对任意扰动集合求最坏情形。

场景误差以该次场景中心为基准，构造当前误差、EMA 和近 36 段均误差。EMA 递推及一般反馈形式为

$$z_j^s = 0.8z_{j-1}^s + 0.2\varepsilon_j^s, \quad z_{-1}^s = 0, \quad (17)$$

$$\begin{aligned} q_j^s &= a_j^q + G_{\nu_q(j)}^q \psi_{\rho_q(j)}^s, & r_j^s &= a_j^r + G_{\nu_r(j)}^r \psi_{\rho_r(j)}^s, \\ E_{j+1}^s &= a_j^E + G_{\nu_E(j)}^E (\varepsilon_j^s, z_j^s)^\top. \end{aligned} \quad (18)$$

截止下标由决策权限决定：未来原合同使用所属日开始前的误差特征，修订合同使用所属六小时块开始前的特征，库存请求可使用当前段误差及 EMA。合同特征接近 36 段均误差、EMA 的顺序组成二维列向量；日或块内共享两个增益，增益逐分量限定在负 10 至 10。当前已锁定的合同不是自由变量。

命题 7 (A：仿射策略的因果性) 给定规划根节点可测的系数，若特征仅由其合法截止时刻的信息构成，则该策略满足非预见性。

证明. 相同历史前缀给出相同特征，根节点拟合系数已经确定，所以对应决策一致；可测函数的仿射组合仍可测。库存特征可以比合同特征多含当前净需求，实现先承诺后响应的权限区分。 $\square$

该策略类没有纳入未来尚未发布的官方预报创新，后者在实际发布后由重新规划使用。**模型内定位 (B)：**上层最优性对应给定经验轨迹、特征和增益界内的受限仿射类；场景内可行不直接推出全部外样本可行，实际供需由执行器追索保证。

5.4 上层场景线性规划及继续价值

将式(18)代入各场景，以合同上图和紧急电量上图表示费用。目标是有限等权场景上的样本平均近似 [6, 7]：

$$\begin{aligned} \min \quad & \frac{1}{S} \sum_{s=1}^S \left[\sum_{j=0}^{H-1} p_j^s (F_j^s + 5e_j^s) + \Psi_a(E_H^s) \right], \\ & F_j^s \geq 1.5r_j^s - 0.5q_j^s, \quad F_j^s \geq 0.5r_j^s + 0.5q_j^s, \\ & e_j^s \geq 0, \quad e_j^s \geq n_j^s - r_j^s + \eta(E_{j+1}^s - E_j^s), \\ & e_j^s \geq n_j^s - r_j^s + (E_{j+1}^s - E_j^s)/\eta, \\ & E_{j+1}^s \in \mathcal{X}(E_j^s), \quad E_0^s = E_a, \quad q_j^s, r_j^s \geq 0, \\ & -10 \leq G^q, G^r, G^E \leq 10. \end{aligned} \quad (19)$$

还须加入机制规定的合同固定等式、式(18)及已抵达真实终点时的库存等式。日前机制始终保持最终合同等于原合同。选择线性末值时，继续费用取

$$\Psi_a(E) = -\lambda_a E, \quad \lambda_a = Q_{0.25}(p_{H-\min(144,H):H}^{1:S})/\eta. \quad (20)$$

这一近似按未来低价补能的边际费用给库存估值，用于抑制短时域末点的过度放电倾向。滚动域到达真正评价终点时令末值系数归零，并由上层硬等式和实际执行可达集合管理末端。该目标是规划评分，正式账单始终按真实交付费逐段求和，不把虚拟末值记入现金节省。

每日 0、6、12、18 时重规划，每次覆盖从当前时刻起的 72 小时，并在真正终点截断。这与微网模型预测控制的滚动时域一致 [1, 2]，但合同变量始终受式(2)约束，不会在十分钟尺度上重新签约。0 时落实当日原合同，合法修订节点落实剩余日内合同，未来场景动作作为价值计算参考。问题二和问题四日前机制的六小时重算不产生额外签约权限。

6 凸 Markov 动态规划：用库存价值响应当前净需求

合同按上一节锁定之后，电池看见眼前缺口，需要判断当前库存中的单位电量是现在用来少买紧急电，还是留到后面。下层把该子问题写成有限状态 Markov 决策过程 [9]，并以凸分段线性值函数作近似动态规划 [10]。斜率合并用来计算下确界卷积，而不必对每个库存点另解一次线性规划。

6.1 状态压缩和固定名义合同

每次上层求解后，以其未来名义合同作为本次下层计算的固定序列。将该次经验场景在每段按净需求三分位分成三箱，当前观测经相同阈值映射为标签。于是下层近似状态为

$$(E_t, K_t), \quad K_t \in \{0, 1, 2\}. \quad (21)$$

相邻场景标签的计数形成时变转移矩阵。每个单元加三分之一伪计数，每行总伪计数为 1:

$$P_j(k, k') = \frac{N_j(k, k') + 1/3}{\sum_{\ell=0}^2 N_j(k, \ell) + 1}. \quad (22)$$

箱内净需求采用该箱场景经验分布，空箱退回全部场景；价格使用箱内条件均价。这是每次规划重建的经验 Markov 近似，并非从全年资料拟合的固定平稳链。箱内价格压缩不保留剩余净需求与价格相关性的全部信息。到下一合法节点，重新求上层 LP、更新名义合同和经验链。

6.2 Bellman 方程与观测顺序

用下一段转移概率组合继续费用，令容量域之外的值为正无穷。固定合同费用与当前库存选择无关，可从该次电池子问题中略去。箱内净需求先被观察，电池再决策，因此值函数采用先最小化再组内平均：

$$W_{j,k}(x) = \sum_{k'=0}^2 P_j(k, k') V_{j+1,k'}(x), \quad (23)$$

$$V_{j,k}(E) = \frac{1}{|B_{j,k}|} \sum_{s \in B_{j,k}} \min_{x \in \mathcal{X}(E)} \{5\bar{p}_{j,k} e(r_j, x - E; n_j^s) + W_{j,k}(x)\}. \quad (24)$$

空箱时使用全部场景作为组内集合。将净需求先取均值再最小化会改变信息顺序，并非式(24)。当前候选在一般滚动末点采用线性末值，源码另提供凸割包络作为辅助分支；本次 DP 遇到给定终点时，源码使用有限绝对值罚

$$V_{H,k}(E) = 1000|E - 6000|. \quad (25)$$

该有限罚提供末端价值引导，真实末端由执行器的硬可达集合保证；精确终点性质由命题 5 给出。

6.3 Bellman 凸性保持定理

令库存消耗量为当前库存减下一库存，定义扩展实值即时函数

$$\begin{aligned} h_{j,k,n}(y) &= 5\bar{p}_{j,k} \max\{0, n - r_j - \eta y, n - r_j - y/\eta\} + \mathcal{I}_{[-\eta M, M/\eta]}(y), \\ (W \square h)(E) &= \inf_{x+y=E} \{W(x) + h(y)\}. \end{aligned} \quad (26)$$

指标函数在集合内为 0、集合外为正无穷；在 W 中同样加入容量域指标。于是每个组内最小化都是下确界卷积。

定理 1 (A: 凸分段线性保持) 若继续函数和即时函数在各自非空有限闭区间上为有限凸分段线性函数，并在域外取正无穷，则其下确界卷积在可行域上为闭凸分段线性函数；有限非负概率平均仍保持该结构。因此式(24)从声明的凸终端函数向后递推时保持凸分段线性。

证明. 即时成本凸性由命题 3 及区间指标得到。对两个可行库存及其最优分配，取相同权重的凸组合仍满足总量等式和区间约束；函数凸性给出组合分配成本不超过原最优成本的加权和，因此最优值凸。有限闭区间保证可行最小值达到；PWL 函数的上图为多面体，将联合上图按总量等式投影仍是多面体，故值函数闭且 PWL。最后，有限个这样的函数按非负概率求和仍凸且 PWL。完整域条件及归纳见附录。 $\square$

该论证也可由凸分析中部分最小化与下确界卷积的上图投影性质理解 [11, 12]。 **推论:** 值函数斜率随库存单调不减；把额外库存带来的继续费用减少称为边际收益时，其值为负斜率，单调不减。这里不要求严格单调，也不宣称整个域上库存收益始终为正，特别是终点惩罚附近。

6.4 斜率合并与连续动作

命题 8 (A: 斜率合并算法) 对有限区间凸 PWL 函数，下确界卷积可由两组非降斜率作有序合并，同时保留每段横向长度构造。两函数从左端点开始分配总增量。先使用边际费用较低的线段；凸性保证各函数内部斜率有序，全局合并仍遵守各自前缀。附录以小量交换给出完整证明。该方法把逐库存重复求 LP 化为线段合并，是下层计算的结构依据。

实现把值函数存储在 321 个库存节点上，卷积后回到网格作分段线性插值。凸函数的弦插值在节点间位于被插值函数上方，但这一局部关系不构成原随机问题的成本界。执行时，用当前净需求对应箱的价格均值，比较价值折点、即时缺口折点和物理边界，得到连续库存请求：

$$x^{\text{req}} \in \arg \min_{x \in \mathcal{X}(E)} \{5\bar{p}_{j,k} e(r, x - E; n) + \widetilde{W}_{j,k}(x)\}. \quad (27)$$

实际候选包括网格节点、当前库存和两个缺口折点，控制输入不限制为 321 个离散值。随后执行器施加真正终点可达投影；投影后的动作不宣称仍是带收缩区间目标的重新精确最小解。321 是数值存储参数，数学凸性来自函数结构。

7 预报融合、价格不确定性与信息价值

官方预报携带近期气象信息，历史预测保留本地光伏的日内形态。二者按共同发布时点融合，再与负荷预测相减形成净需求，能够在交易权限不变的条件下调整购电中心。融合系数进入当前预测，也进入历史残差重建，保证场景中的误差口径一致。

7.1 问题三：按发布覆盖范围融合光伏预报

每次官方发布给出后续 24 小时预测。将发布前已观测的光伏值作为起点，与 24 个小时端点组成插值节点，映射到十分钟交付段。设插值并作非负截断后的官方预测为 $\hat{v}^O$ ，历史预测为 $\hat{v}^H$ 。当前统一候选取

$$\hat{v}_{a+j|a} = \beta \hat{v}_{a+j|a}^O + (1 - \beta) \hat{v}_{a+j|a}^H, \quad \beta = 0.75. \quad (28)$$

式(28)用于已发布预报覆盖的交付段，覆盖之外沿用历史预测；净需求预测为负荷减光伏再除以 6。代码中的额外官方误差修正系数为零。历史预测仍包含已观测日内误差的衰减修正，不能将额外修正为零理解为全部日内修正关闭。问题二及问题四日前机制不使用官方预报，融合系数在这两种机制中不改变预测。

针对是否使用 0 时以外预报的问题，本文保留题给 6、12、18 时发布作为调度输入，在相应节点重新评价剩余合同；是否改变交付量由总费用模型决定。对于题目尚未提供的新发布时点，其价值应在保持交易权限不变的条件下，用带真实发布时间戳的预测作配对回放。现有汇总体现当前发布机制与融合配置的联合表现，不为某个新增时点赋予具体节省额。

7.2 融合的二阶误差解释

命题 9 (A：双预测融合的均方误差) 设两种预测误差具有有限二阶矩，分别记为 e_O 和 e_H 。令

$$A = \mathbb{E}[e_O^2], \quad B = \mathbb{E}[e_H^2], \quad C = \mathbb{E}[e_O e_H], \quad D = A + B - 2C. \quad (29)$$

则融合误差的均方误差及其在单位区间内的最优权重满足

$$M(\beta) = \beta^2 A + (1 - \beta)^2 B + 2\beta(1 - \beta)C, \quad (30)$$

$$D = \mathbb{E}[(e_O - e_H)^2] \geq 0,$$

$$\beta^* = \text{proj}_{[0,1]} \left(\frac{B - C}{D} \right), \quad D > 0. \quad (31)$$

证明. 融合误差等于两误差的同权线性组合，平方取期望得到式(30)。展开为关于权重的二次函数，其二阶导数为 $2D$ ；对正 D 令一阶导数为零并投影到可行区间即可。若 $D = 0$ ，两误差几乎处处相同，各权重给出相同均方误差。□

该结果解释为何两种信息的误差互补可以支持融合。A、B、C 是二阶矩，无需假定两预测无偏。本文的 0.75 来自已有候选配置，既不由未知总体矩估计为理论最优，也不将预测均方误差最小等同于实际购电费用最小；后者还受分时价、紧急费、效率和库存价值共同决定。

7.3 问题四：成对价格场景与动作后结算

问题四将同一历史块的净需求误差和价格误差成对叠加到当前预测。上层逐场景计价，保留该块中两者的联合变化；下层使用净需求箱内的价格均值，在有限场景下保持三状态库存反馈。日前机制锁定当日合同，日内机制按题目允许的节点修订合同。

当前候选的合同特征仍为净需求均误差与 EMA，价格通过场景目标和箱内均价进入决策；实际交付电价在电池动作后用于结算。这一信息时序与将当前真实价格提前提供给控制器的模型不同。

7.4 信息与跨日自由度的精确模型关系

命题 10 (A: 信息集合与策略集合) 若两个精确优化问题的外生联合分布、物理条件、结算、控制权限及真实初末约束相同，且多信息策略可以忽略新增信息，则

$$\mathcal{I}_1 \subseteq \mathcal{I}_2 \implies \Pi(\mathcal{I}_1) \subseteq \Pi(\mathcal{I}_2) \implies J^*(\mathcal{I}_2) \leq J^*(\mathcal{I}_1). \quad (32)$$

证明. 将少信息策略原样用于较丰富信息集，其每个决策仍可实施，故策略集合包含；对相同目标取下确界即可。□

命题 11 (A: 跨日可行域包含关系) 对于同一精确模型和真正终点，将各中间午夜库存固定为 6,000 kWh 的策略集合包含于允许库存连续的策略集合：

$$\Pi_{\text{daily}} \subseteq \Pi_{\text{cross}}, \quad J_{\text{cross}}^* \leq J_{\text{daily}}^*. \quad (33)$$

额外午夜等式给出该包含关系。这两项性质说明信息和跨日自由度在精确模型中的作用。实际场景、仿射类与 Markov 近似形成的闭环费用由轨迹评价；固定 72 小时与预报融合的年度结果据此作为数值实验报告。

8 年度结果与机制验证

8.1 比较对象、配置与证据口径

评价覆盖 2025 年 2 月 1 日至 12 月 31 日 334 日，共 48,096 个十分钟段。比较统一真实初末库存 6,000 kWh，中间午夜连续。1 月作为预测族选择、开发与公共初始化阶段；正式期初状态可由 1 月保持库存 6,000 kWh 的公共可行策略衔接，1 月费用不计入这 334 日账单。

本次远端年度队列报告 113 项运行全部完成，统一候选采用固定 72 小时、28 日成熟历史窗口、7 条等权场景、EMA 系数 0.2、增益界负 10 至 10、321 个库存节点、三状态 Markov 反馈和线性末值。官方预报融合权重为 0.75，额外官方误差修正为零。

比较定位. 该配置已列入候选清单，本文报告其年度比较后的统一结果。一月冻结记录中最终挑战模型采用条件场景与另一组参数，故本候选的年度结果属于候选比较，不按独立留出的年度测试或一月最终选定模型表述。表4的上一版为各任务同初末库存的跨日 Markov 策略，比较保持该基线身份。

证据来源. 问题二具有完整本地 JSON/NPZ 轨迹，本文独立重算账单并检查 48,096 段物理约束，结果与远端汇总一致。问题三及问题四两种机制采用本地同步的远端复核

摘要；其摘要报告物理审计通过，原始逐段缓存与完整审计登记尚未包含在当前数据包中。因此，这三项支持年度汇总比较，逐日区间、指定时段表及费用分解均不从旧轨迹转用。

8.2 四种机制的年度费用

表 4 统一候选与上一版同任务费用比较（万元）

机制	上一版	72 小时融合候选	节省	降幅
问题二	1,384.42	1,381.79	2.62	0.189%
问题三	1,346.18	1,335.24	10.94	0.813%
问题四日前	1,465.46	1,459.24	6.22	0.425%
问题四日内	1,425.21	1,411.73	13.48	0.946%

2025 年 2 月至 12 月 334 日，报告口径均为初末库存 6000 kWh。问题二经本地完整轨迹重算；其余三项按远端复核汇总报告。统一候选属于年度候选比较，对照为上一版跨日 Markov 策略。

年度账单的共同方向是四种机制均较上一版下降。固定价日内机制与变动价日内机制分别节省 10.94 万元和 13.48 万元，是本组比较最清晰的费用收益；日前两种机制分别节省 2.62 万元和 6.22 万元。各机制是不同交易环境下的独立评价，其金额不相加为一个微网的实际全年节省。

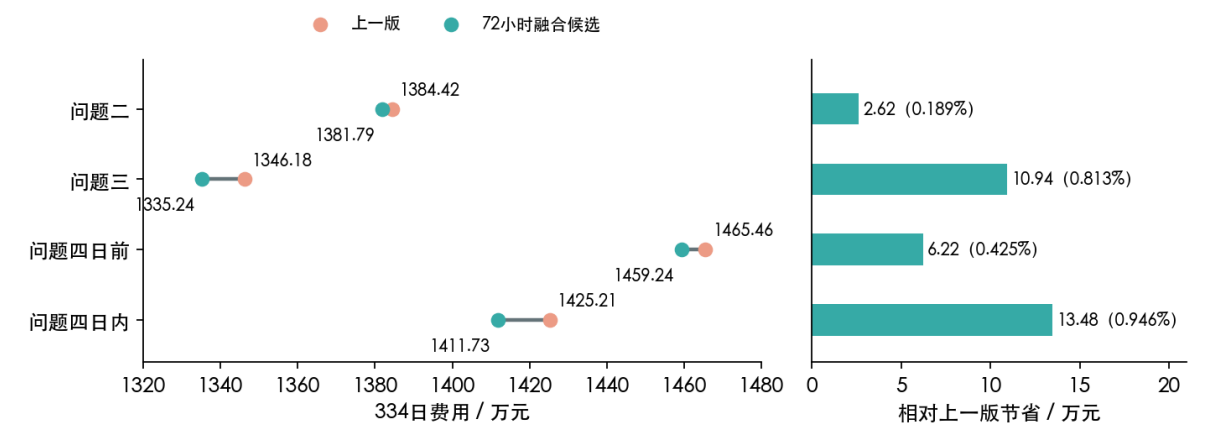


图 5 年度费用与相对上一版节省：在各自相同交易机制内比较

日前机制未使用官方预报，因此其结果说明固定视域及相应场景重构的闭环表现；日内机制同时改变视域与预报融合。当前四项汇总未将这两个因素逐一固定，日内差额按联合配置效果解释。相同 H 下仅改变融合系数的完整轨迹，才可进一步量化融合的独立贡献。

8.3 问题二：节省主要来自紧急购电费

问题二原合同与有效合同相同，调整费为零。旧策略与新候选的普通购电费用分别为 1,319.7126 万元、1,319.7423 万元，紧急购电费用分别为 64.7044 万元、62.0522 万元。二者相减得到

$$\Delta C = \Delta C_{\text{plan}} + \Delta C_{\text{emergency}} = 296.61 - 26,521.61 = -26,225.00. \quad (34)$$

式中差额按新候选减上一版、单位元。普通购电支出增加 296.61 元，换来紧急购电费用下降 26,521.61 元，使总费用下降 26,225.00 元。该费用分解把结果与控制目标连接起来：在普通购电支出基本相当的情况下，跨日安排改变了高倍率紧急补购的费用。

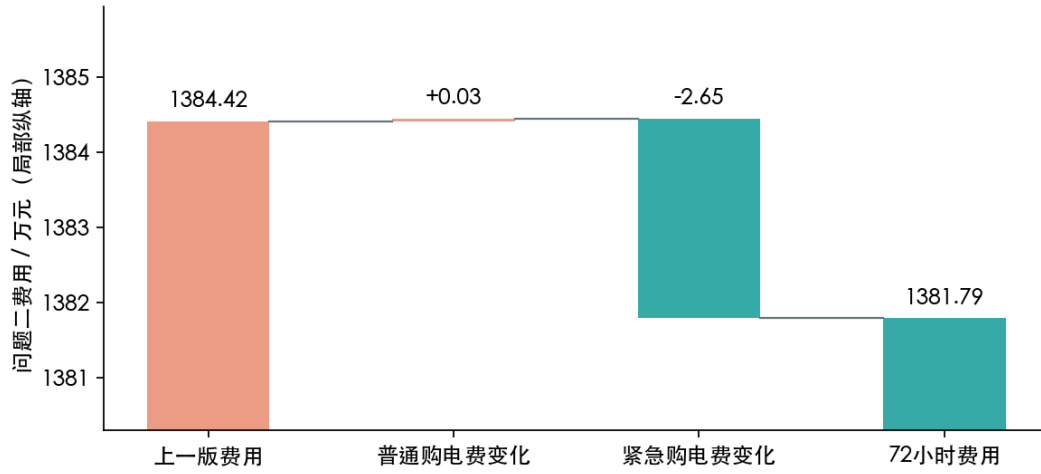


图 6 问题二账单分解：普通购电与紧急购电共同决定净节省

费用分解识别的是结算项目变化，而不是单独固定合同、电池和预测后的因果分量。问题二附录指定日期表直接来自这套新轨迹，四小时块分别累计充电与放电；两列同时为正可由块内不同时段的双向动作产生。

8.4 问题二：时间稳定性与适用范围

按日期对齐新旧策略，定义日费用差及年度累计差：

$$\Delta_d = C_d^{\text{old}} - C_d^{\text{new}}, \quad \bar{\Delta} = \frac{1}{334} \sum_{d=1}^{334} \Delta_d, \quad S_{\text{yr}} = \sum_{d=1}^{334} \Delta_d. \quad (35)$$

问题二日均节省 78.52 元。用 3、7、14 日连续移动块作 10,000 次重采样，保留每个块内的时序关联，并以均值的 2.5% 和 97.5% 分位数构造区间 [14]。三种块长的日均区间分别为 $[-122.81, 295.64]$ 、 $[-101.14, 288.44]$ 和 $[-74.14, 285.42]$ 元。

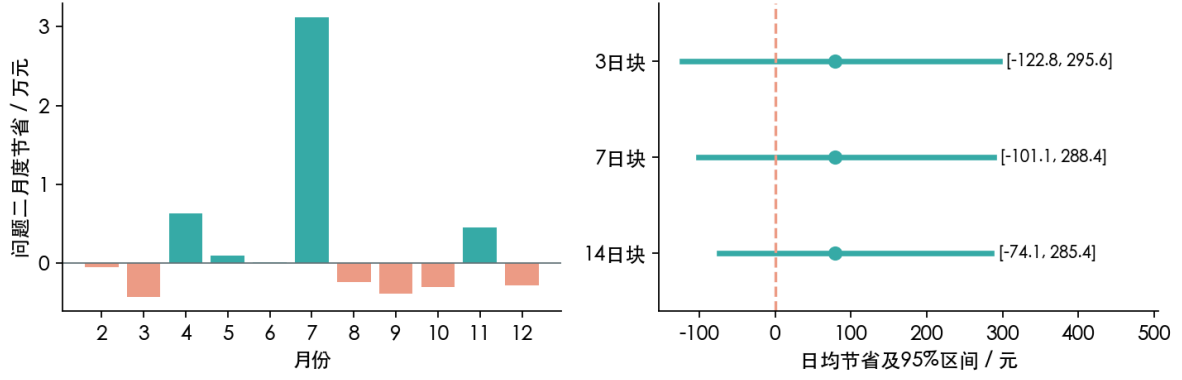


图 7 问题二月度费用差与移动块区间：展示收益随时间的分布

334 日中费用差为正 156 日、负 177 日，按 10^{-6} 元容差计持平 1 日；11 个月中 5 个月累计节省。三种块长区间均包含零，因此本组证据支持当年累计账单下降，并呈现日、月尺度上的波动。连续库存允许在一个日期补能、在后续日期使用，运行期费用是本模型的直接目标。该图承担时间稳定性诊断，不把候选比较的当年节省扩展为跨年度保证。

8.5 物理闭环与真正终点核验

问题二完整轨迹逐段满足母线平衡、库存递推及单向充放电，库存保持在 1,200 至 10,800 kWh 内，功率不超过 5,000 kW，午夜库存连续，真正首末库存均为 6,000 kWh。所有账单在执行后由原合同、有效合同、紧急量与实际价格重新计算，线性末值不计入现金费用。图表使用同一轨迹与同一费用函数，使年度收益有明确的物理可行性基础。

9 性能边界、参数与方法评价

9.1 完美信息参照的严格含义

令事后参照已知全部真实净需求和价格，按普通交付价购电，同时保持相同电池条件和真正初末库存。对任何可实施策略，保留其电池动作与弃电，令参照普通购电为有效合同加紧急购电，则

$$q_t^{\text{PI}} = r_t + e_t, \quad p_t(r_t + e_t) \leq p_t\{\phi(q_t, r_t) + 5e_t\}. \quad (36)$$

因为 $\phi(q, r) = r + 0.5|r - q|$ 且价格非负，该构造费用不高于原策略，并维持母线平衡。于是逐路径有 $L_{\text{PI}}(\omega) \leq C_{\pi}(\omega)$ ；对同一分布取期望并优化因果策略，得到

$$\mathbb{E}[L_{\text{PI}}] \leq J_{\text{causal}}^* \leq \mathbb{E}[C_{\pi}]. \quad (37)$$

它说明完美信息和普通价交易提供一个理论参照。参照距离同时涉及未来信息、交易渠道、有限场景、策略限制与值函数近似，不等于在线算法最优性差距。本文保留该数学关系，不以旧候选的数值距离标注新候选。

9.2 数值参数与辅助价值模型

表 5 当前统一配置及各参数的职责

参数	在当前实现中的作用
72 小时	固定核心长度，真正年末按剩余时间截断。
28 日、7 条轨迹	近期成熟历史窗口与等权时间覆盖的场景预算。
0.75 融合权重	已发布官方预报与历史预测的组合；只作用于官方机制。
0.2 EMA、增益界 $[-10, 10]$	构造误差记忆，并限定上层仿射响应幅度。
321 库存节点	存储、插值凸值函数；实际动作包含连续折点。
三状态	按净需求三分位分箱，用伪计数构造经验转移。
线性末值、倍率 1	按末段场景价格四分位估计继续库存价值。

这些参数共同定义可复现的当前候选。321 点网格的作用是数值存储，7 条轨迹与三状态体现给定计算预算下的表示选择；其数值不由凸性定理唯一决定。

代码还实现了条件权重、场景缩减、统一误差状态和增强反馈，用于另行候选比较。当前年度统一配置采用等权场景、原两维净需求仿射特征与 M0 反馈，故本文把主要论证集中于这一实际执行链。辅助 SDDP 割的下界性质属于两小时、阶段独立噪声的粗模型，证明放入附录；在固定 72 小时的非午夜边界，完整继续状态还含未交付合同，既有午夜库存割不直接作为该边界的继续值。

9.3 模型评价与复核范围

本文通过费用凸性和单向消元，将购电、紧急追索和电池状态统一进线性约束；通过因果特征保证承诺时序；通过凸价值反馈与可达投影实现实时响应和真实末端衔接。这些结构分别对应可解释优化、合法信息使用和物理执行。

算法的适用条件为单电池、连续功率、非负电价、免费无上限弃电及题目给定交易权限。增加电池退化、并网容量或改变退购结算，需要重新写出相应约束和费用。当前新增结果的范围为同年度候选比较；问题三与问题四的原始缓存同步后，可按附录流程扩展逐段复核、指定时段表与稳定性诊断。

10 结论

本文将微网四问组织为同一跨日随机控制系统，以固定 72 小时稳定规划视野，以合法预报融合更新净需求中心，以因果合同和凸库存价值反馈连接提前承诺与实时动作。确定性问题的费用为 35,126.95 元；年度统一候选的四机制费用为 1,381.79、1,335.24、1,459.24、1,411.73 万元，相较上一版同任务结果分别节省 2.62、10.94、6.22、13.48 万元。问题二完整轨迹进一步将净收益定位到紧急购电费的下降。凸性、信息时序与终点可达性共同构成这一调度框架的理论基础。

AI 工具使用说明

本次文稿重写使用生成式人工智能辅助理论整理、代码审阅、数据复核与排版。问题二新增统计由本地脚本读取完整轨迹重新计算，问题三和问题四年度数值引用远端复核汇总；生成内容与计算证据的对应关系记录于支撑文件。

参考文献

- [1] Parisio A, Rikos E, Glielmo L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813–1827. DOI:10.1109/TCST.2013.2295737.
- [2] Parisio A, Rikos E, Glielmo L. Stochastic model predictive control for economic/environmental operation management of microgrids: An experimental case study[J]. Journal of Process Control, 2016, 43: 24–37. DOI:10.1016/j.jprocont.2016.04.008.
- [3] Bhattacharya A, Kharoufeh J P, Zeng B. Managing energy storage in microgrids: A multi-stage stochastic programming approach[J]. IEEE Transactions on Smart Grid, 2018, 9(1): 483–496. DOI:10.1109/TSG.2016.2618621.
- [4] Ben-Tal A, Goryashko A, Guslitzer E, Nemirovski A. Adjustable robust solutions of uncertain linear programs[J]. Mathematical Programming, 2004, 99(2): 351–376. DOI:10.1007/s10107-003-0454-y.
- [5] Goulart P J, Kerrigan E C, Maciejowski J M. Optimization over state feedback policies for robust control with constraints[J]. Automatica, 2006, 42(4): 523–533. DOI:10.1016/j.automatica.2005.08.023.
- [6] Kleywegt A J, Shapiro A, Homem-de-Mello T. The sample average approximation method for stochastic discrete optimization[J]. SIAM Journal on Optimization, 2002, 12(2): 479–502. DOI:10.1137/S1052623499363220.
- [7] Shapiro A, Dentcheva D, Ruszczyński A. Lectures on Stochastic Programming: Modeling and Theory[M]. 2nd ed. Philadelphia: SIAM, 2014.
- [8] Kaut M, Wallace S W. Evaluation of scenario-generation methods for stochastic programming[J]. Pacific Journal of Optimization, 2007, 3(2): 257–271.
- [9] Puterman M L. Markov Decision Processes: Discrete Stochastic Dynamic Programming[M]. New York: Wiley, 1994.
- [10] Powell W B. Approximate Dynamic Programming: Solving the Curses of Dimensionality[M]. 2nd ed. Hoboken: Wiley, 2011.
- [11] Rockafellar R T. Convex Analysis[M]. Princeton: Princeton University Press, 1970.
- [12] Boyd S, Vandenberghe L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004. <https://web.stanford.edu/~boyd/cvxbook/>.
- [13] HiGHS Development Team. HiGHS: High performance software for linear optimization[EB/OL]. <https://highs.dev/>, 2026-09-11.
- [14] Künsch H R. The jackknife and the bootstrap for general stationary observations[J]. Annals of Statistics, 1989, 17(3): 1217–1241. DOI:10.1214/aos/1176347265.

A 凸结构与可达性的完整证明

A.1 合同上图与单向消元的等价性

式(6)的两分支在合同相等处连续。引入 F 满足两条上图不等式后，对任意原可行解可令 F 等于两仿射式最大值，得到同费用的 LP 可行解；反向对任一 LP 可行解，将 F 降至其最大值不会破坏约束，非负价格下费用不增。因此两个问题的最优值相同。价格为零时 LP 可能返回松弛 F ，但账单重新计算仍按原合同函数。

命题 2 的母线构造也可逐项验证。保持外网合同与紧急电不变，令

$$w' = w + (1 - \eta^2)u. \quad (38)$$

则

$$r + e + d' - c' - w' = r + e + d - c - w = n. \quad (39)$$

若弃电另有限制，上式新增富余未必能被消纳；若存在循环奖励，去除循环未必维持目标值。因此本文的单向存在性以正文条件为边界。对单向动作，净母线耗电在正增量为增量除效率，在负增量为效率乘增量，得到凸折线 g 。给定 g 后，最小非负紧急电等于净需求加 g 减合同的正部，弃电由平衡恢复，分别得到式(10)。其关于库存增量的凸性无需净需求非负。

A.2 终点区间的充要构造及递归可行性

假设下一库存为 x ，目标为 z ，尚余 N 步。若 N 为正且 x 不高于 z ，取每次库存增加量为

$$\delta_i = (z - x)/N, \quad i = 1, \dots, N. \quad (40)$$

区间左界保证此值不超过最大充电库存增量；所有中间点位于 x 与 z 之间。若 x 高于 z ，取同样等步长的负增量，区间右界保证其绝对值不超过最大放电库存减量。恢复母线动作并加入必要外网补电或弃电，构造了满足所有当前约束的完整后缀。

执行器每步将下一库存置于剩余可达域，所以选定动作后仍存在到达目标的后缀。以剩余步数递减归纳，可得真实终点的递归可行性。初始状态本身必须可达，才有非空交集；本文 6,000 至 6,000 的初末设置显然具有保持库存的可行路径。对中间午夜实施同样目标是每日闭合对照的额外约束，不是实际电池的物理必需条件。

B Bellman 与斜率合并的完整证明

B.1 真性、闭性和分段线性的域条件

设两个函数分别在有限闭区间上有限、连续、凸 PWL，区间外为正无穷。它们均为真函数，即不取负无穷且不恒为正无穷，并具有多面体上图。可行总量域为两区间的 Minkowski 和，对可行总量，分配集合是非空紧集，连续目标在其上达到最小值。设上图变量为 z ，则卷积上图可表示为

$$\{(E, z) : \exists x, y, u, v, E = x + y, u \geq W(x), v \geq h(y), z \geq u + v\}. \quad (41)$$

这是有限多条线性不等式和等式描述的多面体的线性投影，故仍是多面体。它是闭凸函数的上图，因此卷积为闭凸 PWL。若只给一般真凸函数而没有这里的紧域和 PWL 条件，则不应跳过闭性及最小值达到的讨论。

在 Bellman 中，W 是下一标签值函数的有限概率平均，加入容量区间指标后保留上述性质；h 的有限域是单步放能变量区间。每个当前标签组内的函数对所有物理库存均可行，因为外网追索无上限，库存可保持不变。有限平均后的共同域仍是容量区间。线性末值、最大仿射割与有限绝对值罚均为凸 PWL，故向后归纳成立。

若对精确理论 DP 采用硬终端单点指标，须按扩展实值函数限制有效域，并处理逐步扩展的可达区间。本稿实现采用有限罚和独立执行投影，正文没有将二者混称为硬终端精确 DP。

B.2 斜率合并的资源交换证明

设 f 在区间 $[a, A]$ 、g 在 $[b, B]$ 上为凸 PWL。f 各线段斜率、长度依次为

$$(s_i, \ell_i)_{i=1}^m, \quad s_1 \leq \dots \leq s_m, \quad \ell_i > 0, \quad (42)$$

g 对应斜率长度为

$$(t_j, d_j)_{j=1}^n, \quad t_1 \leq \dots \leq t_n, \quad d_j > 0. \quad (43)$$

总量为 a 加 b 加 L 时，相当于分别给 f 和 g 分配非负横向增量，增量之和为 L。函数增值等于其非降边际斜率沿前缀的积分。

将两组斜率稳定有序合并，保留每段原长度；相同斜率任意交织，但保持各自原有次序。取合并序列前 L 长度，得到两函数各自的合法前缀：若某函数的后段被选择而更早段未被选择，则后段斜率不低于前段，有序选择可以把相同或更低斜率前段先纳入，且不增成本。

为避免交换破坏原函数的前缀约束，先把各线段视为可独立取量、上限为该段长度的资源，暂时放松前缀约束。现设这一放松问题的另一分配在尚未使用较低边际成本片段时使用了较高片段。选择不超过两片段可用长度的正小量，从高边际片段移到低边际片段，保持总增量且不增费用；严格高低时费用严格降低。重复交换可得到放松问题的有序合并最优分配。前段已经证明该排序分配同时满足两函数各自的前缀约束，因此放松问题的最优解也在原问题可行，二者最优值相同。因而合并前 L 长度的成本达到最小值。

令合并后第 k 段斜率、长度为 v 和 h，则卷积折点与函数值从左端初值开始累计：

$$z_0 = a + b, \quad F_0 = f(a) + g(b), \quad z_k = z_{k-1} + h_k, \quad F_k = F_{k-1} + v_k h_k. \quad (44)$$

两组斜率已排序时合并复杂度与线段总数成正比；源码直接排序全部段，成本还包括排序。数学正确性来自有序边际资源分配，而不是网格点个数。

B.3 网格插值与动作折点

对凸函数，任意相邻网格节点的弦不低于真实函数。代码在每次递推后插值回网格，再把网格折线视为下一次卷积的输入，所以它对当前插值函数执行斜率合并，而不是对原始完整随机过程进行精确运算。

对固定当前净需求，即时成本折点来自以下三处（仅保留处于可行区间者）：

$$x = E, \quad x = E - (n - r)/\eta, \quad x = E - \eta(n - r). \quad (45)$$

与继续值折点和区间边界合并后，相邻候选之间目标为仿射，故最小值可在端点取得。这解释了动作候选为何允许连续值。终点可达投影使用新的收缩区间，源码未重新完整枚举该区间目标，因此不据此宣称投影动作具有精确 Bellman 最优性。

C 辅助 SDDP 割的有效性证明

固定两小时聚合模型、有限时域和阶段独立噪声分布。以下结论以阶段 LP 有限且满足强对偶、所需追索子问题可行、已有割包络为有效全局下界为条件。设真实下一阶段值为 V ，其已有割包络为下界。给定输入状态及一个有限噪声实现，替换下一值后得到近似阶段 LP 最优值。由于下界替换降低或保持目标，对所有可行输入状态有

$$\underline{Q}_t(X, \xi) \leq Q_t(X, \xi). \quad (46)$$

在访问状态处，强对偶给出输入状态固定等式的对偶乘子。该乘子是近似阶段值的次梯度，因此

$$\underline{Q}_t(\bar{X}, \xi) + \beta_t(\xi)^\top (X - \bar{X}) \leq \underline{Q}_t(X, \xi) \leq Q_t(X, \xi). \quad (47)$$

按固定有限噪声概率完整求和，形成期望阶段值的全局仿射下界：

$$\sum_{\xi} p_{\xi} \left\{ \underline{Q}_t(\bar{X}, \xi) + \beta_t(\xi)^\top (X - \bar{X}) \right\} \leq \sum_{\xi} p_{\xi} Q_t(X, \xi). \quad (48)$$

辅助末端费用为零且阶段成本非负，零割有效；从终端向前归纳得到全部生成割的有效性。这里支撑的是**近似阶段值**，不能把访问点近似目标误当真实继续值。状态中尚未交付合同必须保留；根节点签约会重置合同，因此根割可以进一步仅依赖库存。

阶段独立性使固定噪声概率无需再依赖被省略的历史状态；若用于历史相关的正式预测过程，必须扩充状态或改变模型，不能沿用本证明直接获得原问题下界。当前 SDDP 割作为末值代理的效果由开发期实际闭环账单判断。

D 固定时域执行与统计复核

D.1 完整闭环算法

1. 声明真实评价区间和初末库存，加载已确定的预测族及当前候选配置，保持 1 月公共初始化与 2 月期初衔接。
2. 在 0、6、12、18 时确定核心末点，取当前绝对时段加 432 与真正终点中的较小值；截断可见实际数据与未来未发布预报。
3. 在合法官方覆盖内按 0.75 与 0.25 融合预测；以相同预测方法重建成熟历史残差，按时间覆盖选取最多 7 条并赋予等权。
4. 解因果仿射场景 LP。0 时落实完整当日原合同；日内官方机制在合法节点更新剩余有效合同，日前机制保持原承诺。

5. 以当次名义合同构造三箱 Markov 转移。未来函数按概率平均，箱内成本作下确界卷积，插值回 321 个库存节点。
6. 每十分钟先观察净需求，求连续库存请求，再投影到物理与真实终点可达交集；恢复单向电池动作、紧急购电和弃电。
7. 电池动作确定后按实际交付价格结算。库存传至下一段与下一日，评价终点回到 6,000 kWh。
8. 保存全部合同、发布记录、电池动作、库存和费用；按日期对齐比较轨迹，重算账单与统计，登记图表来源。

D.2 问题二移动块重采样

给定 334 个日费用差，对块长 b 使用全部 $334 - b + 1$ 个非环绕连续块。每次等概率抽取足够多的块，拼接并保留前 334 日，计算均值；重复 10,000 次。三种块长分别以随机种子 20260912 初始化，块内保留原日期顺序。均值百分位区间乘 334 给出累计区间，数据文件同时保存日差、月差及正负日数。该方法近似保留短期依赖，对候选后验选择和跨年度分布变化不提供额外修正。

D.3 源码与证据的一致性检查

当前候选在源码中满足等权场景、EMA 为 0.2、增益界 10 及三箱条件，实际调用原上层仿射规划与三状态 Markov 反馈分支。新场景工厂提供固定时域和融合预测；增强状态分支属于其他候选。每次运行的源文件哈希、配置、实际日期和末端约束应与结果一起保存。

问题二的年度成本、月差、费用分解和指定日期表均从同一完整 NPZ 重算。问题三及问题四当前保留汇总证据层级，完整缓存复核时须检查：原合同未被后续发布覆盖、最终合同按相对原计划计费、各段母线平衡和库存递推、充放电功率与互斥、午夜连续、真实年末库存，以及新旧轨迹日期和价格相同。检查通过后再生成对应日期表与移动块统计。

E 问题二指定日期结果

下列四个指定日期均来自固定 72 小时候选的完整轨迹。购电量列为当段合同电量，紧急购电单列；年度费用包含两者按规定倍率的结算。中间各日首末库存按跨日状态自然衔接，只有真正评价首末固定为 6,000 kWh。

表 6 问题二 2025-03-20 购电与电池动作（电量 kWh，费用元）

时间段	合同电量	时间段	合同电量	时间段	合同电量
10:00–10:10	0.00	12:00–12:10	570.07	14:00–14:10	0.00
16:00–16:10	510.46	18:00–18:10	697.51	20:00–20:10	0.00
四小时段	充电量	放电量	四小时段	充电量	放电量
00:00–04:00	1,789.67	136.83	04:00–08:00	1,231.50	5,482.33
08:00–12:00	5,461.70	2,777.38	12:00–16:00	5,082.70	531.46
16:00–20:00	246.22	5,484.52	20:00–24:00	3,362.59	3,042.09
日初库存		8,340.47	日末库存		4,403.39
全天合同		62,542.21	全天紧急购电		33.83
全天实际费用					39,136.98

表 7 问题二 2025-06-21 购电与电池动作（电量 kWh，费用元）

时间段	合同电量	时间段	合同电量	时间段	合同电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	258.65	18:00–18:10	0.00	20:00–20:10	0.00
四小时段	充电量	放电量	四小时段	充电量	放电量
00:00–04:00	559.16	256.70	04:00–08:00	805.95	1,809.83
08:00–12:00	5,673.77	0.00	12:00–16:00	4,000.00	567.00
16:00–20:00	217.53	5,157.72	20:00–24:00	7,927.93	2,633.45
日初库存		3,761.14	日末库存		9,444.05
全天合同		41,479.71	全天紧急购电		0.00
全天实际费用					23,549.69

表 8 问题二 2025-09-23 购电与电池动作（电量 kWh，费用元）

时间段	合同电量	时间段	合同电量	时间段	合同电量
10:00–10:10	0.00	12:00–12:10	155.19	14:00–14:10	0.00
16:00–16:10	485.35	18:00–18:10	733.27	20:00–20:10	0.00
四小时段	充电量	放电量	四小时段	充电量	放电量
00:00–04:00	2,074.83	17.46	04:00–08:00	1,052.20	5,666.49
08:00–12:00	5,121.42	1,896.62	12:00–16:00	3,800.98	671.67
16:00–20:00	310.85	5,793.59	20:00–24:00	7,906.13	2,517.18
日初库存		8,417.82	日末库存		8,254.22
全天合同		67,544.32	全天紧急购电		70.56
全天实际费用					43,248.33

表 9 问题二 2025-12-21 购电与电池动作（电量 kWh，费用元）

时间段	合同电量	时间段	合同电量	时间段	合同电量
10:00–10:10	0.00	12:00–12:10	919.99	14:00–14:10	0.00
16:00–16:10	810.90	18:00–18:10	704.07	20:00–20:10	0.00
四小时段	充电量	放电量	四小时段	充电量	放电量
00:00–04:00	2,259.09	92.53	04:00–08:00	2,178.13	1,925.87
08:00–12:00	5,422.06	6,716.57	12:00–16:00	7,282.85	2,754.00
16:00–20:00	7.91	5,721.12	20:00–24:00	7,883.89	2,873.49
日初库存		8,137.60	日末库存		8,353.06
全天合同		97,471.34	全天紧急购电		0.00
全天实际费用					63,036.51

表 10 问题二四个指定日期的连续紧急购电区间（kWh）

2025-03-20	2025-06-21	2025-09-23	2025-12-21
时间段	电量	时间段	电量
16:00–16:20	23.81	无	0.00
16:30–16:40	8.56	–	–
17:10–17:20	1.46	–	–
全天合计	33.83	全天合计	0.00

逐段紧急购电量大于 10^{-6} kWh 时计入区间；仅合并连续十分钟段。区间量与原轨迹全天总量之差均小于 10^{-6} kWh，差额为低于阈值的数值残差。

F 支撑文件列表与完整调度程序

文件	职责
revised-main.tex	竞赛正文、年度比较及结果解释。
technical-appendix.tex	完整证明、执行流程与问题二指定结果。
artifacts/remote-reported-summary.json	同步的远端四机制汇总，保留原证据身份。
artifacts/q2-new-analysis.json	由完整轨迹重算的费用分解、月差与配对区间。
artifacts/result-registry.json	数值来源与证据等级登记。
paper/result-claims.json	正文数值与登记项对应。
src/build_evidence.py	重算统计、生成表格与图形。
src/write_paper.py	生成正文与证明附录。
model-source/	与当前仓库一致的完整调度源程序，全文如下。
review/	独立控制、凸优化、证明与代码审查。

模型源程序与数据的原始位置由支撑包 README 说明。复算采用固定配置、既定预测选择文件和原始数据；独立检验按已保存轨迹重新计费。以下按模块列出完整调度代码，第三方线性规划求解器作为运行依赖安装。

F.1 费用结算与确定性调度

```

1  """Convex sample-average planning and causal battery execution; all energy in kWh."""
2  from functools import lru_cache
3  import numpy as np
4  from scipy.optimize import linprog
5  from scipy.sparse import eye, hstack, vstack, kron, csr_matrix, diags
6
7  @lru_cache(maxsize=256)
8  def matrices(T,S,adjust,eta):
9      I=eye(T,format='csr'); Z=csr_matrix((T,T)); R=kron(np.ones((S,1)),I,format='csr')
10     # -q + c - d - emergency <= -net demand
11     Aub=hstack([-R,R,-R,csr_matrix((S*T,T)),-eye(S*T)],format='csr')
12     D=eye(T)-diags(np.ones(T-1),-1,shape=(T,T))
13     Aeq=hstack([Z,-eta*I,I/eta,D,csr_matrix((T,S*T))],format='csr')
14     if adjust:
15         Aub=hstack([Aub,csr_matrix((S*T,2*T))],format='csr')
16         Aeq=hstack([Aeq,csr_matrix((T,2*T))],format='csr')
17         Aeq=vstack([Aeq,hstack([I,Z,Z,Z,csr_matrix((T,S*T)),-I,I]),format='csr')
18     return Aub,Aeq
19
20 def solve(net,prices,initial=6000.,terminal=6000.,base=None,refund=True,eta=.9,
21         power=5000.,emin=1200.,emax=10800.,penalty=5.,hard=False):
22     S,T=net.shape; adjust=base is not None; p=prices.mean(0)
23     Aub,Aeq=matrices(T,S,adjust,eta)
24     obj=np.r_[np.zeros(T) if adjust else p,np.zeros(2*T),np.zeros(T),penalty*prices.ravel()/S]
25     if adjust: obj=np.r_[obj,1.5*p,(-.5 if refund else .5)*p]
26     rhs=np.zeros(T); rhs[0]=initial
27     if adjust: rhs=np.r_[rhs,base]
28     bounds=[(0,None)]*T+[(0,power/6)]*(2*T)+[(emin,emax)]*T+[(0,0 if hard else None)]*(S*T)
29     bounds[4*T-1]=(terminal,terminal)
30     if adjust: bounds += [(0,None)]*(2*T)
31     res=linprog(obj,A_ub=Aub,b_ub=-net.ravel(),A_eq=Aeq,b_eq=rhs,bounds=bounds,method='highs',
32               options={'dual_feasibility_tolerance':1e-8,'primal_feasibility_tolerance':1e-8})
33     if not res.success: raise RuntimeError(f'LP failed: {res.message}')
34     x=res.x; q=x[:T]; c=x[T:2*T]; d=x[2*T:3*T]
35     # Analytic removal of any numerical simultaneous charge/discharge, preserving SOC.
36     remove=np.minimum(c,d/eta**2); c=c-remove; d=d-eta**2*remove
37     state=np.r_[initial,initial+np.cumsum(eta*c-d/eta)]
38     em=np.maximum(net+c-d-q,0)
39     if adjust:
40         cost=float(np.sum(1.5*p*np.maximum(q-base,0)+(-.5 if refund else .5)*p*np.maximum(base-q,0))+
41                 penalty*np.mean(np.sum(prices*em,axis=1)))
42     else: cost=float(p@q+penalty*np.mean(np.sum(prices*em,axis=1)))
43     return {'q':q,'c':c,'d':d,'state':state,'expected_cost':cost,'lp_objective':float(res.fun),
44           'eq_residual':float(np.max(np.abs(Aeq*x-rhs))),
45           'ineq_violation':float(max(0,np.max(Aub@x+net.ravel()))),
46           'duality_gap':float(abs(res.fun-(np.dot(rhs,res.eqlin.marginals)+np.dot(-net.ravel(),res.ineqlin.mar
47           ↪ ginals)+
48                 sum(lo*m for (lo,hi),m in zip(bounds,res.lower.marginals) if lo is not None)+
49                 sum(hi*m for (lo,hi),m in zip(bounds,res.upper.marginals) if hi is not None)))})
50 def execute_slot(q,net,E,c_plan,d_plan,t,controller='fixed',eta=.9,power=5000.,emin=1200.,emax=10800.,terminal=6000.):
51     ↪ 000.):
52     if controller=='fixed': c,d=c_plan,d_plan
53     elif controller=='greedy':
54         balance=q-net
55         nextE=E+eta*min(max(balance,0),power/6,(emax-E)/eta)-min(max(-balance,0),power/6,eta*(E-emin))/eta
56         # Backward reachable terminal set prevents last-day battery depletion borrowing.
57         left=143-t
58         lower=max(emin,terminal-left*eta*power/6)
59         upper=min(emax,terminal+left*power/(6*eta))
60         nextE=np.clip(nextE,lower,upper)
61         c=max(0,(nextE-E)/eta); d=max(0,(E-nextE)*eta)
62     else: raise ValueError(controller)
63     nextE=E+eta*c-d/eta
64     balance=q+d-c-net
65     emergency=max(-balance,0); spill=max(balance,0)
66     return c,d,nextE,emergency,spill
67
68 def settlement(q,r,emergency,price,refund=True,penalty=5.):
69     planned=price*q
70     increase=1.5*price*np.maximum(r-q,0)
71     reduction=(-.5 if refund else .5)*price*np.maximum(q-r,0)

```

```
71     emerg=penalty*price*emergency
72     return {'planned_cost':float(planned.sum()),'increase_cost':float(increase.sum()),
73            'reduction_net_cost':float(reduction.sum()),'emergency_cost':float(emerg.sum()),
74            'total_cost':float((planned+increase+reduction+emerg).sum())}
```

F.2 合法信息预测

```

1 """Absolute-time, as-of forecasts and fully matured multiday residual blocks."""
2 import numpy as np
3
4 FAMILIES=('lag1','lag7','mean7','trend7')
5
6 def base_forecast(data,key,asof,targets,family):
7     day=asof//144; td=targets//144; tt=targets%144; x=data[key]
8     if day<1: raise ValueError('No preceding complete day')
9     if family=='mean7': return x[max(0,day-7):day].mean(0)[tt]
10    lag=1 if family=='lag1' else 7
11    hd=np.minimum(td-lag,day-1);hd=np.maximum(0,hd)
12    if family=='trend7':
13        older=np.maximum(0,np.minimum(td-14,day-1))
14        return np.maximum(0,2*x[hd,tt]-x[older,tt])
15    return x[hd,tt].copy()
16
17 def select_families(data):
18     out={}
19     for key in ['load','pv','price']:
20         scores={f:float(np.mean([np.abs(base_forecast(data,key,d*144,np.arange(d*144,(d+1)*144),f)-data[key][d])
21             ↪ .mean(0) for d in range(14,31)])) for f in FAMILIES}
22         out[key]={ 'selected':min(scores,key=scores.get), 'january_mae':scores}
23     return out
24
25 def forecast_as_of(data,selection,asof,targets,official,variable):
26     """Only completed days, already observed intraday values, latest release."""
27     pred={k:base_forecast(data,k,asof,targets,selection[k]['selected']) for k in ['load','pv','price']}
28     day,t=divmod(asof,144)
29     # A conservative observed-error correction, with a six-hour fading memory.
30     # Coefficient 0.8 is a declared shrinkage, unchanged across annual runs.
31     if t:
32         past=np.arange(max(day*144,asof-36),asof)
33         fade=np.exp(-(targets-asof)/36.)
34         for k in ['load','pv','price']:
35             err=data[k].ravel()[past]-base_forecast(data,k,day*144,past,selection[k]['selected'])
36             pred[k]=np.maximum(0,pred[k]+.8*err.mean()*fade)
37     official_covered=0
38     if official:
39         issue=(asof//36)*36; d,k=divmod(issue//36,4)
40         anchor=data['pv'].ravel()[issue-1] if issue else 0.
41         endpoints=np.arange(25)*6+issue
42         mask=(targets+1<=issue+144)&(targets+1>=issue)
43         pred['pv'][mask]=np.interp(targets[mask]+1,endpoints,np.r_[anchor,data['forecast'][d,k]])
44         official_covered=int(mask.sum())
45     if not variable: pred['price']=data['day_price'][targets%144]
46     pred['price']=np.maximum(.001,pred['price'])
47     pred['net']=(pred['load']-pred['pv'])/6
48     pred['official_covered_slots']=official_covered
49     return pred
50
51 def scenarios(data,selection,asof,end,official,variable>window=28,count=7):
52     targets=np.arange(asof,end);T=len(targets); day,t=divmod(asof,144)
53     # A block must be mature at the current decision time, including its last target.
54     eligible=[h for h in range(max(7,day-window-3),day) if h*144+t+T<=asof]
55     eligible=eligible[-window:]
56     if not eligible: raise ValueError('No fully mature historical blocks')
57     if count and len(eligible)>count:
58         eligible=[eligible[j] for j in np.linspace(0,len(eligible)-1,count).round().astype(int)]
59     current=forecast_as_of(data,selection,asof,targets,official,variable)
60     ns=[];ps=[]
61     for h in eligible:
62         issue=h*144+t; ht=np.arange(issue,issue+T)
63         old=forecast_as_of(data,selection,issue,ht,official,variable)
64         actual=(data['load'].ravel()[ht]-data['pv'].ravel()[ht])/6
65         ns.append(current['net']+actual-old['net'])
66         ps.append(np.maximum(.001,current['price']+data['price'].ravel()[ht]-old['price']) if variable else
67             ↪ current['price'])
68     return np.array(ns),np.array(ps),{'as_of':asof,'end':end,'history_days':eligible,
69         'latest_training_target_exclusive':max(eligible)*144+t+T,
70         'official_covered_slots':current['official_covered_slots']}

```

F.3 因果仿射规划与凸库存反馈

```

1  """Causal affine-policy SAA MPC and one-dimensional convex Markov DP."""
2  from pathlib import Path
3  from types import SimpleNamespace
4  import sys
5  import numpy as np
6  from scipy.sparse import coo_matrix, csr_matrix, vstack, eye
7  sys.path.insert(0, str(Path(__file__).resolve().parents[1]/'vendor'))
8  import highsipy
9
10 ETA=.9; M=5000/6; EMIN=1200.; EMAX=10800.; INITIAL=6000.
11
12 def lp_solve(obj,A,b,bounds,solver='ipm'):
13     A=A.tocsr();p=highspy.HighsLp();p.num_col=len(obj);p.num_row=len(b)
14     p.col_cost=np.asarray(obj);p.col_lower=np.array([a if a is not None else -np.inf for a,z in
15     ↪ bounds]);p.col_upper=np.array([z if z is not None else np.inf for a,z in bounds])
16     p.row_lower=np.full(len(b),-np.inf);p.row_upper=np.asarray(b)
17     p.a_matrix_.format_=highspy.MatrixFormat.kRowwise;p.a_matrix_.start_=A.indptr;p.a_matrix_.index_=A.indices;p
18     ↪ .a_matrix_.value_=A.data
19     h=highspy.Highs();h.setOptionValue('output_flag',False);h.setOptionValue('threads',1);h.setOptionValue('solv
20     ↪ er',solver);h.passModel(p);h.run()
21     if h.getModelStatus()!=highspy.HighsModelStatus.kOptimal:raise
22     ↪ RuntimeError(h.modelStatusToString(h.getModelStatus()))
23     x=np.array(h.getSolution().col_value)
24     return SimpleNamespace(x=x,fun=h.getObjectiveValue(),violation=float(max(0,np.max(A@x-b))))
25
26 def features(net):
27     err=net-net.mean(0);ema=np.zeros_like(err);avg=np.zeros_like(err);z=np.zeros(len(net))
28     for t in range(net.shape[1]):
29         z=.8*z+.2*err[:,t];ema[:,t]=z;avg[:,t]=err[:,max(0,t-35):t+1].mean(1)
30     return err,ema,avg
31
32 def affine_plan(net,prices,asof,E,base=None,adjust=False,affine=True,tail_price=0.,terminal=None,tail_cuts=None,
33 ↪ daily_closed=False,solver='ipm'):
34     """Node-time affine recourse. Future contracts use ONLY pre-release errors.
35
36     Scenario states are feasible in-sample; unseen paths are projected physically.
37     Future official forecast innovations are omitted from this restricted policy
38     class, while actual current releases are used by the scenario producer.
39     """
40     S,T=net.shape;K=S*T;clock=np.arange(asof,asof+T); day0=asof//144
41     blocks=clock//36-clock[0]//36;B=int(blocks.max()+1);days=clock//144-day0;D=int(days.max()+1)
42     err,ema,avg=features(net)
43     # a_q, a_r, a_E, g_q, g_r, g_E, emergency, delivered contract charge.
44     aq=0; ar=T; ae=2*T;gq=3*T;gr=gq+2*D;ge=gr+2*B;em=ge+2*B;bill=em+K;theta=bill+K;N=theta+(S if tail_cuts else
45     ↪ 0)
46
47     def expr(which):
48         rows=[];cols=[];vals=[];constant=np.zeros(K)
49         for s in range(S):
50             for j,ts in enumerate(clock):
51                 row=s*T+j;di=days[j];bi=blocks[j]
52                 if which=='Q' and di==0 and base is not None:
53                     constant[row]=base[j];continue
54                 if which=='Q': a=aq+j;g=gq+2*di;reveal=(ts//144)*144-asof
55                 elif which=='R': a=ar+j;g=gr+2*bi;reveal=(ts//36)*36-asof
56                 else:a=ae+j;g=ge+2*bi;reveal=j+1
57                 rows.append(row);cols.append(a);vals.append(1.)
58             if affine and reveal>0:
59                 values=(err[s,j],ema[s,j]) if which=='E' else (avg[s,reveal-1],ema[s,reveal-1])
60                 for k,val in enumerate(values):rows.append(row);cols.append(g+k);vals.append(val)
61         return coo_matrix((vals,(rows,cols)),shape=(K,N)).tocsr(),constant
62     Q,q0=expr('Q'); R,r0=expr('R') if adjust else (Q,q0);X,x0=expr('E')
63     if adjust:
64         firstblock=np.tile(clock%144<36,S).astype(float)
65         from scipy.sparse import diags
66         keep=diags(firstblock);R=keep@Q+(eye(K)-keep)@R;r0=firstblock*q0+(1-firstblock)*r0
67     shift=coo_matrix((np.ones(S*(T-1)),(np.concatenate([np.arange(s*T+1,(s+1)*T) for s in
68     ↪ range(S)]),np.concatenate([np.arange(s*T,(s+1)*T-1) for s in range(S)]))),shape=(K,K)).tocsr()
69     Delta=X-shift@X; delta0=np.zeros(K);delta0[np.arange(S)*T]==-E
70     U=coo_matrix((np.ones(K),(np.arange(K),em+np.arange(K))),shape=(K,N)).tocsr()
71     F=coo_matrix((np.ones(K),(np.arange(K),bill+np.arange(K))),shape=(K,N)).tocsr()
72     mats=[-Q,-R,X,-X,Delta,-Delta,ETA*Delta-R-U,Delta/ETA-R-U,1.5*R-.5*Q-F,.5*R+.5*Q-F]

```

```

65     rhs=[q0,r0,np.full(K,EMAX),np.full(K,-EMIN),np.full(K,ETA*M)-delta0,np.full(K,M/ETA)+delta0,
66           r0-net.ravel()-ETA*delta0,r0-net.ravel()-delta0/ETA,.5*q0-1.5*r0,-.5*q0-.5*r0]
67     last=np.arange(S)*T+T-1
68     if terminal is not None:
69         mats += [X[last],-X[last]];rhs += [np.full(S,terminal),np.full(S,-terminal)]
70     if daily_closed:
71         mids=np.flatnonzero(np.tile((clock+1)%144==0,S))
72         mats += [X[mids],-X[mids]];rhs += [np.full(len(mids),6000.),np.full(len(mids),-6000.)]
73     if tail_cuts:
74         Theta=coo_matrix((np.ones(S),(np.arange(S),theta+np.arange(S))),shape=(S,N)).tocsr()
75         for alpha,beta in tail_cuts:mats.append(beta*X[last]-Theta);rhs.append(np.full(S,-alpha))
76     obj=np.zeros(N);obj[em:bill]=(5*prices/S).ravel();obj[bill:theta]=(prices/S).ravel()
77     if tail_cuts:obj[theta:]=1/S
78     obj-=tail_price*np.asarray(X[last].mean(0)).ravel()
79     bounds=[(None,None)]*(3*T)+[(-10,10) if affine else (0,0)]*(2*D+4*B)+[(0,None)]*(2*K+(S if tail_cuts else 0))
80     sol=lp_solve(obj,vstack(mats),np.concatenate(rhs),bounds,solver);x=sol.x
81     qs=(Q@x+q0).reshape(S,T);rs=(R@x+r0).reshape(S,T);xs=(X@x).reshape(S,T)
82     prev=np.c_[np.full(S,E),xs[:,-1]];delta=x-prev
83     emer=np.maximum(net-rs+np.maximum(ETA*delta,delta/ETA),0)
84     true=float(np.mean(np.sum(prices*(np.maximum(1.5*rs-.5*qs,.5*rs+.5*qs)+5*emer),axis=1))-tail_price*xs[:,-1].
    ↪ mean())
85     if tail_cuts:true+=float(np.maximum(0,np.max([a+b*xs[:,-1] for a,b in tail_cuts],axis=0)).mean())
86     return {'q':qs.mean(0),'r':rs.mean(0),'a':x[ae:ae+T],'g':x[ge:ge+2*B].reshape(B,2),
87           'center':net.mean(0),'asof':asof,'lp_violation':sol.violation,'objective_gap':abs(true-sol.fun),
88           'scenario_q':qs,'scenario_r':rs,'scenario_state':xs,'forecast_contract_gain':x[gq:gr],
89           'all_gain':x[gq:em],
90           'gain_active_columns':np.asarray(abs(vstack(mats)[: ,gq:em]).sum(axis=0)).ravel()>1e-12}
91
92 def prune_cuts(cuts):
93     """Remove globally inactive lines on [EMIN, EMAX], preserving the envelope."""
94     by_slope={}
95     for a,b in [[0,.0.]]+list(cuts):by_slope[b]=max(a,by_slope.get(b,-np.inf))
96     hull=[];starts=[]
97     for b,a in sorted(by_slope.items()):
98         start=-np.inf
99         while hull:
100             pa,pb=hull[-1];start=(pa-a)/(b-pb)
101             if start>starts[-1]:break
102             hull.pop();starts.pop()
103             if not hull:start=-np.inf
104             hull.append([a,b]);starts.append(start)
105     keep=[line for i,line in enumerate(hull) if starts[i]<=EMAX and (i==len(hull)-1 or starts[i+1]>=EMIN)]
106     probes=np.linspace(EMIN,EMAX,301)
107     assert np.max(np.abs(np.max([a+b*probes for a,b in [[0,.0.]]+list(cuts)],axis=0)-np.max([a+b*probes for a,b
    ↪ in keep],axis=0)))<1e-6
108     return keep
109
110 def execute(E,target,q,net,remaining,closed=False,t=0,final=6000.):
111     lo=max(EMIN,E-M/ETA);hi=min(EMAX,E+ETA*M)
112     if closed: remaining=143-t
113     if final is not None:
114         lo=max(lo,final-remaining*ETA*M);hi=min(hi,final+remaining*M/ETA)
115     if lo>hi+1e-6:raise RuntimeError('Unreachable terminal inventory')
116     nex=float(np.clip(target,lo,hi));delta=nex-E;c=max(delta,0)/ETA;d=max(-delta,0)*ETA
117     balance=q+d-c-net
118     return nex,c,d,max(-balance,0),max(balance,0),abs(nex-target)
119
120 def inf_convolution(grid,V,net_minus_q,price):
121     """Exact convex PL infimal convolution, then interpolate on the state grid.
122
123     min_x V(x)+5p max(0,n-q+g(x-E)). Slopes of two convex PL
124     functions are merged. Interpolation is an upper chord approximation.
125     """
126     a=net_minus_q;ys=np.array(sorted(set([-ETA*M,M/ETA,0.,float(np.clip(a/ETA,-ETA*M,M/ETA)),float(np.clip(a*ETA
    ↪ ,-ETA*M,M/ETA))])))
127     cost=5*price*np.maximum(0,a+np.maximum(-ETA*ys,-ys/ETA))
128     lengths=np.r_[np.diff(grid),np.diff(ys)];slopes=np.r_[np.diff(V)/np.diff(grid),np.diff(cost)/np.diff(ys)]
129     order=np.argsort(slopes,kind='stable');knots=np.r_[grid[0]+ys[0],grid[0]+ys[0]+np.cumsum(lengths[order])]
130     values=np.r_[V[0]+cost[0],V[0]+cost[0]+np.cumsum(lengths[order]*slopes[order])]
131     return np.interp(grid,knots,values)
132
133 class MarkovDP:
134     """Observable net-load bins, conditional price means, continuous actions.
135

```

```

136 Future provisional contracts use the planner's common nominal quantities.
137 Grid value interpolation approximates convex Bellman functions; it is not
138 advertised as SDDP or a global lower bound.
139 """
140 def __init__(self, net, prices, q, tail_price, grid_size=321, terminal=None, tail_cuts=None):
141     S, T = net.shape; self.grid = np.linspace(EMIN, EMAX, grid_size); self.T = T
142     cuts = np.quantile(net, [1/3, 2/3], axis=0).T; self.cuts = cuts
143     labels = (net > cuts[:, 0]).astype(int) + (net > cuts[:, 1]).astype(int)
144     self.future = np.zeros((T, 3, grid_size)); self.price = np.zeros((T, 3))
145     # A finite convex terminal penalty approximates the target in this DP;
146     # hard true-end reachability is enforced separately in execution.
147     v = np.repeat((-tail_price * self.grid)[None, :], 3, axis=0)
148     if tail_cuts: v = np.repeat(np.maximum(0, np.max([a + b * self.grid for a, b in
149 ↪ tail_cuts], axis=0))[None, :], 3, axis=0)
149     if terminal is not None: v = np.repeat((1000 * np.abs(self.grid - terminal))[None, :], 3, axis=0)
150     for j in range(T-1, -1, -1):
151         counts = np.zeros((3, 3))
152         if j < T-1: np.add.at(counts, (labels[:, j], labels[:, j+1]), 1.)
153         P = (counts + 1/3) / (counts.sum(1, keepdims=True) + 1)
154         nxt = P @ v; self.future[j] = nxt; cur = np.empty_like(v)
155         for k in range(3):
156             ids = np.flatnonzero(labels[:, j] == k)
157             if not len(ids): ids = np.arange(S)
158             price = float(prices[ids, j].mean()); self.price[j, k] = price
159             cur[k] = np.mean([inf_convolution(self.grid, nxt[k], net[s, j] - q[j], price) for s in ids], axis=0)
160         v = cur
161     self.initial_values = v
162
163 def action(self, j, E, net, q):
164     k = int(net > self.cuts[j, 0]) + int(net > self.cuts[j, 1]); a = net - q
165     lo = max(EMIN, E - M/ETA); hi = min(EMAX, E + ETA * M)
166     # A convex PL minimum occurs at a value knot or an immediate-cost kink.
167     candidates = np.unique(np.r_[self.grid[(self.grid >= lo) & (self.grid <= hi)], lo, hi, np.clip([E, E - a/ETA, E - a * ETA],
168 ↪ lo, hi)])
169     delta = candidates - E; cost = 5 * self.price[j, k] * np.maximum(0, a + np.maximum(ETA * delta, delta/ETA)) + np.interp(cand
170 ↪ idates, self.grid, self.future[j, k])
171     return float(candidates[np.argmin(cost)])

```


F.4 固定配置与场景生成

```
1 """As-of conditional trajectory measures; no future realization in planning."""
2 from dataclasses import dataclass, asdict
3 import hashlib
4 import json
5 import numpy as np
6 from scipy.spatial.distance import cdist
7 from forecasting import forecast_as_of
8
9
10 @dataclass(frozen=True)
11 class Config:
12     horizon_hours: int = 48
13     window: int = 28
14     scenarios: int = 7
15     alpha: float = .2
16     gain_bound: float = 10.
17     grid: int = 321
18     bins: int = 3
19     scenario_method: str = 'legacy'
20     upper: str = 'legacy'
21     lower: str = 'M0'
22     tail_scale: float = 1.
23     tail: str = 'linear'
24     horizon_mode: str = 'fixed'
25     final: float | None = 6000.
26     ridge: float = 1e-4
27     innovation_count: int = 10
28     fusion_weight: float = 1.
29     official_correction: float = 0.
30
31     def identity(self):
32         return hashlib.sha256(json.dumps(asdict(self), sort_keys=True).encode()).hexdigest()[:16]
33
34
35 def weighted_quantile(values, quantiles, weights):
36     values = np.asarray(values); weights = np.asarray(weights)
37     if np.allclose(weights, weights[0], rtol=0, atol=1e-15):
38         return np.quantile(values, quantiles)
39     order = np.argsort(values, kind='stable'); w = weights[order] / weights.sum()
40     return np.interp(quantiles, np.cumsum(w) - .5*w, values[order])
41
42
43 def reduce_measure(distance, mass, count, max_swaps=20):
44     """Deterministic weighted forward selection plus PAM swaps, not global PAM."""
45     n = len(mass); count = min(count, n)
46     selected = [int(np.argmin(distance @ mass))]
47     closest = distance[:, selected[0]].copy()
48     while len(selected) < count:
49         improvement = mass @ (closest[:, None] - np.minimum(closest[:, None], distance))
50         improvement[selected] = -np.inf
51         chosen = int(np.argmax(improvement))
52         if improvement[chosen] <= 1e-12:
53             break # Duplicate trajectories do not create independent representatives.
54         selected.append(chosen); closest = np.minimum(closest, distance[:, chosen])
55     swaps = 0
56     for _ in range(max_swaps):
57         old = float(mass @ np.min(distance[:, selected], axis=1)); best = old; choice = None
58         for position in range(len(selected)):
59             for replacement in range(n):
60                 if replacement in selected:
61                     continue
62                 candidate = selected.copy(); candidate[position] = replacement
63                 cost = float(mass @ np.min(distance[:, candidate], axis=1))
64                 if cost < best - 1e-10:
65                     best, choice = cost, candidate
66         if choice is None:
67             break
68         selected = choice; swaps += 1
69     selected = np.array(sorted(selected), dtype=int)
70     assignments = np.argmin(distance[:, selected], axis=1)
71     reduced = np.bincount(assignments, weights=mass, minlength=len(selected))
72     positive = reduced > 0
```

```

73     selected, reduced = selected[positive], reduced[positive]
74     reduced /= reduced.sum()
75     transport = float(mass @ np.min(distance[:, selected], axis=1))
76     return selected, reduced, {'transport_distance': transport, 'pam_swaps': swaps}
77
78
79 def state_paths(net_error, price_error, seeds, alpha, variable):
80     """Battery-node state: current n error, its EMA, lagged p error/EMA."""
81     S, T = net_error.shape
82     seeds = np.broadcast_to(seeds, (S, 4))
83     x = np.empty((S, T, 4 if variable else 2))
84     zn = seeds[:, 1].copy(); zp = seeds[:, 3].copy(); ep = seeds[:, 2].copy()
85     for j in range(T):
86         zn = (1-alpha)*zn + alpha*net_error[:, j]
87         x[:, j, :2] = np.c_[net_error[:, j], zn]
88         if variable:
89             x[:, j, 2:] = np.c_[ep, zp]
90             ep = price_error[:, j]
91             zp = (1-alpha)*zp + alpha*ep
92     return x
93
94
95 class ScenarioFactory:
96     def __init__(self, data, selection, official, variable, alpha=.2,
97                  fusion_weight=1., official_correction=0.):
98         self.data, self.selection = data, selection
99         self.official, self.variable, self.alpha = official, variable, alpha
100        self.fusion_weight, self.official_correction = fusion_weight, official_correction
101        self.forecasts = {}; self.blocks = {}
102        n = (data['load']-data['pv']).ravel()/6; p = data['price'].ravel()
103        self.ledger = np.full((len(n), 4), np.nan)
104        en = n[144:]-n[:-144]; ep = p[144:]-p[:-144]
105        zn, zp = en[0], ep[0]
106        for j in range(len(en)):
107            zn = (1-alpha)*zn+alpha*en[j]; zp = (1-alpha)*zp+alpha*ep[j]
108            self.ledger[j+144] = en[j], zn, ep[j], zp
109
110        def seed(self, asof):
111            assert asof > 144
112            seed = self.ledger[asof-1].copy()
113            assert np.isfinite(seed).all()
114            return seed
115
116        def forecast(self, origin, length):
117            key = origin, length
118            if key not in self.forecasts:
119                visible = dict(self.data)
120                for name in ('load', 'pv', 'price'):
121                    visible[name] = self.data[name].copy()
122                    visible[name].ravel()[origin:] = np.nan
123                visible['forecast'] = self.data['forecast'].copy()
124                visible['forecast'].reshape(-1, 24)[origin//36+1:] = np.nan
125                pred = forecast_as_of(visible, self.selection, origin,
126                                     np.arange(origin, origin+length), self.official, self.variable)
127                if self.official and (self.fusion_weight != 1. or self.official_correction != 0.):
128                    targets = np.arange(origin, origin+length)
129                    historical = forecast_as_of(visible, self.selection, origin, targets, False, self.variable)
130                    covered = targets+1 <= origin//36*36+144
131                    adjustment = 0.
132                    if self.official_correction:
133                        errors = []
134                        for past_issue in range(max(144, origin-36)//36*36, origin, 36):
135                            observed = np.arange(max(origin-36, past_issue), min(origin, past_issue+36))
136                            if not len(observed): continue
137                            old = forecast_as_of(visible, self.selection, past_issue, observed, True, self.variable)
138                            errors.extend((visible['pv'].ravel()[observed]-old['pv']).tolist())
139                            adjustment = self.official_correction*float(np.mean(errors)) if errors else 0.
140                    official_pv = np.maximum(0, pred['pv']+adjustment*np.exp(-(targets-origin)/36.))
141                    pred['pv'][covered] =
142                    ↪ self.fusion_weight*official_pv[covered]+(1-self.fusion_weight)*historical['pv'][covered]
143                    pred['net'] = (pred['load']-pred['pv'])/6
144                    assert np.isfinite(pred['net']).all() and np.isfinite(pred['price']).all()
145                    self.forecasts[key] = pred
146            return self.forecasts[key]

```

```

146
147 def block(self, origin, length, asof):
148     assert origin+length <= asof, 'Unripe historical residual block'
149     key = origin, length
150     if key not in self.blocks:
151         past = self.forecast(origin, length); targets = slice(origin, origin+length)
152         en = (self.data['load'].ravel()[targets]-self.data['pv'].ravel()[targets])/6-past['net']
153         ep = self.data['price'].ravel()[targets]-past['price']
154         self.blocks[key] = en, ep
155     return self.blocks[key]
156
157 def build(self, asof, end, config):
158     assert config.alpha == self.alpha, 'Factory and policy EMA parameters differ'
159     assert config.fusion_weight == self.fusion_weight and config.official_correction ==
160     ↪ self.official_correction
161     T = end-asof; day, phase = divmod(asof, 144)
162     origins = np.array([h*144+phase for h in range(7, day)
163         ↪ if h*144+phase+T <= asof][-config.window:], dtype=int)
164     if not len(origins):
165         raise ValueError('No mature historical blocks')
166     pairs = [self.block(int(h), T, asof) for h in origins]
167     rn, rp = np.array([v[0] for v in pairs]), np.array([v[1] for v in pairs])
168     seeds = np.array([self.seed(int(h)) for h in origins]); seed = self.seed(asof)
169     dimension = 4 if self.variable else 2
170     scales = np.maximum(np.std(seeds[:, :dimension], axis=0, ddof=1 if len(seeds)>1 else 0), 1e-8)
171     standardized = seeds[:, :dimension]/scales
172     origin_distance = np.linalg.norm((seeds[:, :dimension]-seed[:dimension])/scales, axis=1)
173     pair_distance = cdist(standardized, standardized)
174     nonzero = pair_distance[pair_distance > 1e-10]
175     bandwidth = float(np.median(nonzero)) if len(nonzero) else 1.
176     if config.scenario_method == 'conditional':
177         logweights = -.5*(origin_distance/bandwidth)**2
178         mass = np.exp(logweights-logweights.max()); mass /= mass.sum()
179     else:
180         mass = np.full(len(origins), 1/len(origins))
181         ns = max(float(np.std(rn)), 1e-8); ps = max(float(np.std(rp)), 1e-8)
182         joint = np.c_[rn/ns, rp/ps] if self.variable else rn/ns
183         distances = cdist(joint, joint)/np.sqrt(joint.shape[1])
184         if config.scenario_method == 'legacy':
185             selected = np.arange(len(origins))
186             if len(selected) > config.scenarios:
187                 selected = np.linspace(0, len(selected)-1, config.scenarios).round().astype(int)
188             weights = np.full(len(selected), 1/len(selected))
189             reduction = {'transport_distance': float(mass @ distances[:, selected].min(1)), 'pam_swaps': 0}
190         elif config.scenario_method in ('medoids', 'conditional'):
191             selected, weights, reduction = reduce_measure(distances, mass, config.scenarios)
192         else:
193             raise ValueError(config.scenario_method)
194     current = self.forecast(asof, T)
195     completion_length = (-end) % 144
196     completion = np.maximum(0, self.forecast(asof, T+completion_length)['net'][T:]) if completion_length else
197     ↪ np.array([])
198     net = current['net'][None, :]+rn[selected]
199     prices = np.maximum(.001, current['price'][None, :]+rp[selected]) if self.variable else
200     ↪ np.tile(current['price'], (len(selected), 1))
201     if config.scenario_method == 'legacy':
202         # Preserve even the floating-point evaluation order of the accepted
203         # generator for the single-factor horizon compatibility experiments.
204         net = np.array([current['net']+(self.data['load'].ravel()[h:h+T]-self.data['pv'].ravel()[h:h+T])/6-s]
205         ↪ self.forecast(int(h), T)['net'] for h in origins[selected]])
206     if self.variable:
207         prices = np.array([np.maximum(.001,
208         ↪ current['price']+self.data['price'].ravel()[h:h+T]-self.forecast(int(h), T)['price']) for h
209         ↪ in origins[selected]])
210     xn = net-current['net']; xp = prices-current['price']
211     states = state_paths(xn, xp, seed, config.alpha, self.variable)
212     history_states = state_paths(rn, rp, seeds, config.alpha, self.variable)
213     tails = rn.sum(1); threshold = weighted_quantile(tails, [.95], mass)[0]
214     metadata = {
215         'as_of': asof, 'end': end, 'horizon_slots': T,
216         'history_origins': origins.tolist(), 'history_days': (origins//144).tolist(),
217         'latest_training_target_exclusive': int(origins.max()+T),
218         'maturity_end': int(origins.max()+T), 'requested_window': config.window,
219         'effective_window': len(origins), 'requested_scenarios': config.scenarios,

```

```

214     'effective_scenarios': len(selected), 'representative_origins': origins[selected].tolist(),
215     'candidate_weights': mass.tolist(), 'scenario_weights': weights.tolist(),
216     'weight_ess': float(1/(mass@mass)), 'bandwidth': bandwidth,
217     'state_seed': seed.tolist(), 'state_seed_observation_cutoff': asof-1,
218     'seed_reference': 'previous-day same-slot forecast, EMA initialized at first valid error',
219     'state_reference': 'fixed current forecast origin; deterministic restart at next planning origin',
220     'scenario_method': config.scenario_method, 'tail95_representative_retained':
    ↪ bool(np.any(tails[selected] >= threshold)),
221     'max_net_residual_retained': bool(np.argmax(rn.max(1)) in selected),
222     'max_price_residual_retained': bool(np.argmax(rp.max(1)) in selected) if self.variable else None,
223     'official_covered_slots': current['official_covered_slots'], **reduction}
224 metadata['outside_core_original_contract_completion'] = {
225     'start': end, 'end': end+completion_length, 'forecast_information_cutoff': asof,
226     'scheduled_release': end//144*144 if completion_length else None,
227     'q_kwh': completion.tolist(), 'feedback_gain': 0.,
228     'rule': 'common nonnegative current-origin forecast; suffix is a causal extension, not optimized
    ↪ delivery',
229     'delivery_cost_in_core_objective': False}
230 return {'net': net, 'prices': prices, 'weights': weights, 'states': states,
231         'history_states': history_states, 'history_net_error': rn,
232         'history_price_error': rp, 'history_seeds': seeds, 'mass': mass,
233         'forecast': current, 'seed': seed, 'origins': origins, 'meta': metadata}

```

F.5 候选控制接口

```

1 """Probability-consistent causal affine planning and convex value feedback."""
2 import time
3 import numpy as np
4 from scipy.sparse import coo_matrix, vstack, eye, diags
5 from scipy.spatial.distance import cdist
6 from control import ETA, M, EMIN, EMAX, lp_solve, inf_convolution, MarkovDP, affine_plan as legacy_affine_plan
7 from nextgen_scenarios import weighted_quantile
8
9
10 def affine_plan(bundle, asof, inventory, config, base=None, adjust=False,
11               tail_price=0., terminal=None, tail_cuts=None):
12     started = time.perf_counter()
13     net, prices, weights = (bundle[k] for k in ('net', 'prices', 'weights'))
14     S, T = net.shape; K = S*T; clock = np.arange(asof, asof+T)
15     if config.upper == 'legacy' and config.alpha == .2 and config.gain_bound == 10. and np.all(weights == 1/S):
16         if tail_cuts and (asof+T) % 144:
17             raise ValueError('Existing SDDP inventory cuts require midnight')
18         answer = legacy_affine_plan(net, prices, asof, inventory, base, adjust, True, tail_price, terminal,
19                                   ↪ tail_cuts)
20         active = answer.pop('gain_active_columns'); gains = answer.pop('all_gain')
21         answer.update({'runtime_seconds': time.perf_counter()-started,
22                       'gain_active_count': int(active.sum()),
23                       'gain_bound_hit_count': int(np.sum((np.abs(gains)>=10*(1-1e-6)) & active)),
24                       'gain_max_abs': float(np.max(np.abs(gains[active])), initial=0),
25                       'contract_information_cutoffs': {'original': (clock//144*144).tolist(), 'effective':
26                                                         ↪ (clock//36*36 if adjust else clock//144*144).tolist()},
27                       'future_partial_contract_completion': 'outside-core suffix fixed to nonnegative origin
28                                                         ↪ forecast, no tail delivery cost'})
29     return answer
30
31 blocks = clock//36-clock[0]//36; B = int(blocks.max()+1)
32 days = clock//144-asof//144; D = int(days.max()+1)
33 if config.upper == 'legacy':
34     error = net-weights@net; ema = np.empty_like(error); avg = np.empty_like(error)
35     z = np.zeros(S)
36     for j in range(T):
37         z = (1-config.alpha)*z+config.alpha*error[:, j]; ema[:, j] = z
38         avg[:, j] = error[:, max(0, j-35):j+1].mean(1)
39     energy_features = np.stack((error, ema), axis=2)
40     previous_features = np.stack((avg, ema), axis=2)
41 elif config.upper == 'state':
42     energy_features = bundle['states'].copy()
43     previous_features = energy_features.copy()
44     if energy_features.shape[-1] == 4:
45         # Battery state has p_(t-1); next release may use the newly disclosed p_t.
46         ep = prices-bundle['forecast']['price']
47         zp = np.full(S, bundle['seed'][3])
48         for j in range(T):
49             zp = (1-config.alpha)*zp+config.alpha*ep[:, j]
50             previous_features[:, j, 2:] = np.c_[ep[:, j], zp]
51             scale = max(float(np.std(bundle['history_net_error'])),
52                         ↪ 1e-8)/max(float(np.std(bundle['history_price_error'])), 1e-8)
53             energy_features[:, :, 2:] *= scale; previous_features[:, :, 2:] *= scale
54         energy_features -= np.einsum('s,std->td', weights, energy_features)
55         previous_features -= np.einsum('s,std->td', weights, previous_features)
56     else:
57         raise ValueError(config.upper)
58 Fdim = energy_features.shape[2]
59 aq = 0; ar = T; ae = 2*T; gq = 3*T; gr = gq+Fdim*D
60 ge = gr+Fdim*B; em = ge+Fdim*B; bill = em+K; theta = bill+K
61 N = theta+(S if tail_cuts else 0)
62
63 def expression(which):
64     rows, cols, values = [], [], []; constant = np.zeros(K)
65     for s in range(S):
66         for j, ts in enumerate(clock):
67             row = s*T+j; di = days[j]; bi = blocks[j]
68             if which == 'Q' and di == 0 and base is not None:
69                 constant[row] = base[j]; continue
70             if which == 'Q':
71                 intercept, gain, reveal = aq+j, gq+Fdim*di, ts//144*144-asof
72             elif which == 'R':
73                 intercept, gain, reveal = ar+j, gr+Fdim*bi, ts//36*36-asof

```

```

69         else:
70             intercept, gain, reveal = ae+j, ge+Fdim*bi, j+1
71             rows.append(row); cols.append(intercept); values.append(1.)
72             if reveal > 0:
73                 feature = energy_features[s, j] if which == 'E' else previous_features[s, reveal-1]
74                 for k, value in enumerate(feature):
75                     rows.append(row); cols.append(gain+k); values.append(value)
76             return coo_matrix((values, (rows, cols)), shape=(K, N)).tocsr(), constant
77
78 Q, q0 = expression('Q'); R, r0 = expression('R') if adjust else (Q, q0)
79 X, _ = expression('E')
80 if adjust:
81     first = np.tile(clock % 144 < 36, S).astype(float); keep = diags(first)
82     R = keep@Q+(eye(K)-keep)@R; r0 = first*q0+(1-first)*r0
83 shift = coo_matrix((np.ones(S*(T-1)),
84     (np.concatenate([np.arange(s*T+1, (s+1)*T) for s in range(S)]),
85     np.concatenate([np.arange(s*T, (s+1)*T-1) for s in range(S)]))), shape=(K, K)).tocsr()
86 Delta = X-shift@X; delta0 = np.zeros(K); delta0[np.arange(S)*T] = -inventory
87 U = coo_matrix((np.ones(K), (np.arange(K), em+np.arange(K)))), shape=(K, N)).tocsr()
88 C = coo_matrix((np.ones(K), (np.arange(K), bill+np.arange(K)))), shape=(K, N)).tocsr()
89 mats = [-Q, -R, X, -X, Delta, -Delta, ETA*Delta-R-U, Delta/ETA-R-U,
90     1.5*R-.5*Q-C, .5*R+.5*Q-C]
91 rhs = [q0, r0, np.full(K, EMAX), np.full(K, -EMIN),
92     np.full(K, ETA*M)-delta0, np.full(K, M/ETA)+delta0,
93     r0-net.ravel()-ETA*delta0, r0-net.ravel()-delta0/ETA,
94     .5*q0-1.5*r0, -.5*q0-.5*r0]
95 last = np.arange(S)*T+T-1
96 if terminal is not None:
97     mats += [X[last], -X[last]]; rhs += [np.full(S, terminal), np.full(S, -terminal)]
98 if tail_cuts:
99     if (asof+T) % 144:
100         raise ValueError('Existing inventory-only SDDP cuts require a midnight contract-reset interface')
101     Theta = coo_matrix((np.ones(S), (np.arange(S), theta+np.arange(S))), shape=(S, N)).tocsr()
102     for intercept, slope in tail_cuts:
103         mats.append(slope*X[last]-Theta); rhs.append(np.full(S, -intercept))
104     obj = np.zeros(N); obj[em:bill] = (5*prices*weights[:, None]).ravel()
105     obj[bill:theta] = (prices*weights[:, None]).ravel()
106     if tail_cuts:
107         obj[theta:] = weights
108     obj -= tail_price*np.asarray(weights@X[last]).ravel()
109     bounds = [(None, None)]*(3*T)+[(-config.gain_bound, config.gain_bound)]*(Fdim*(D+2*B))+[(0, None)]*(2*K+(S if
110     ↪ tail_cuts else 0))
111     sol = lp_solve(obj, vstack(mats), np.concatenate(rhs), bounds)
112     qs = (Q@sol.x+q0).reshape(S, T); rs = (R@sol.x+r0).reshape(S, T)
113     xs = (X@sol.x).reshape(S, T); delta = xs-np.c_[np.full(S, inventory), xs[:, :-1]]
114     emergency = np.maximum(net-rs+np.maximum(ETA*delta, delta/ETA), 0)
115     bills = np.sum(prices*(np.maximum(1.5*rs-.5*qs, .5*rs+.5*qs)+5*emergency), axis=1)
116     recomputed = float(weights@(bills-tail_price*xs[:, -1]))
117     if tail_cuts:
118         recomputed += float(weights@np.maximum(0, np.max([a+b*xs[:, -1] for a, b in tail_cuts], axis=0)))
119     gains = sol.x[gq:em]; bound_hit = np.abs(gains) >= config.gain_bound*(1-1e-6)
120     # Inactive root-node gains do not count toward the binding-frequency denominator.
121     active_columns = np.asarray(abs(vstack(mats)[:, gq:em]).sum(axis=0)).ravel() > 1e-12
122     return {'q': weights@qs, 'r': weights@rs, 'scenario_q': qs, 'scenario_r': rs,
123         'scenario_state': xs, 'lp_violation': sol.violation,
124         'objective_gap': abs(recomputed-sol.fun), 'objective': sol.fun,
125         'runtime_seconds': time.perf_counter()-started,
126         'gain_active_count': int(active_columns.sum()),
127         'gain_bound_hit_count': int(np.sum(bound_hit & active_columns)),
128         'gain_max_abs': float(np.max(np.abs(gains[active_columns])), initial=0)),
129         'contract_information_cutoffs': {'original': (clock//144*144).tolist(),
130         ↪ 'effective': (clock//36*36 if adjust else clock//144*144).tolist()}},
131         'future_partial_contract_completion': 'outside-core suffix fixed to nonnegative forecast from this
132         ↪ origin; suffix delivery costs omitted by inventory-tail truncation'}
133
134 class WeightedMarkovDP:
135     def __init__(self, net, prices, q, weights, tail_price, grid_size=321, bins=3, terminal=None,
136     ↪ tail_cuts=None):
137         self.legacy = None
138         if bins == 3 and np.allclose(weights, 1/len(weights), rtol=0, atol=1e-15):
139             self.legacy = MarkovDP(net, prices, q, tail_price, grid_size, terminal, tail_cuts)
140             return
141         S, T = net.shape; self.grid = np.linspace(EMIN, EMAX, grid_size)

```

```

140     self.cuts = np.array([weighted_quantile(net[:, j], np.arange(1, bins)/bins, weights) for j in range(T)])
141     labels = np.sum(net[:, :, None] > self.cuts[None], axis=2)
142     self.future = np.empty((T, bins, grid_size)); self.price = np.empty((T, bins))
143     v = np.tile(terminal_values(self.grid, tail_price, terminal, tail_cuts), (bins, 1))
144     for j in range(T-1, -1, -1):
145         counts = np.zeros((bins, bins))
146         if j < T-1:
147             np.add.at(counts, (labels[:, j], labels[:, j+1]), S*weights)
148             P = (counts+1/bins)/(counts.sum(1, keepdims=True)+1)
149             nxt = P@v; self.future[j] = nxt; cur = np.empty_like(v)
150             for k in range(bins):
151                 ids = np.flatnonzero(labels[:, j] == k)
152                 if not len(ids): ids = np.arange(S)
153                 w = weights[ids]/weights[ids].sum(); price = float(w@prices[ids, j]); self.price[j, k] = price
154                 cur[k] = w@np.array([inf_convolution(self.grid, nxt[k], net[s, j]-q[j], price) for s in ids])
155             v = cur
156     self.initial_values = v
157
158     def action(self, j, E, net, q, state=None):
159         if self.legacy is not None:
160             return self.legacy.action(j, E, net, q)
161         k = int(np.sum(net > self.cuts[j]))
162         return minimize_action(self.grid, self.future[j, k], E, net-q, self.price[j, k])
163
164
165     def terminal_values(grid, price, terminal, cuts):
166         if terminal is not None:
167             return 1000*np.abs(grid-terminal)
168         if cuts:
169             return np.maximum(0, np.max([a+b*grid for a, b in cuts], axis=0))
170         return -price*grid
171
172
173     def minimize_action(grid, future, E, net_minus_q, price):
174         lo = max(EMIN, E-M/ETA); hi = min(EMAX, E+ETA*M); a = net_minus_q
175         candidates = np.unique(np.r_[grid[(grid >= lo) & (grid <= hi)], lo, hi, np.clip([E, E-a/ETA, E-a*ETA], lo,
176         ↪ hi)])
177         delta = candidates-E
178         values = 5*price*np.maximum(0, a+np.maximum(ETA*delta, delta/ETA))+np.interp(candidates, grid, future)
179         return float(candidates[np.argmin(values)])
180
181     def nonnegative_kernel(points, query, scales, prior):
182         distances = cdist(np.atleast_2d(query)/scales, points/scales)**2
183         # Unit bandwidth in legal historical standard deviations, fixed before development.
184         logits = -.5*distances+np.log(np.maximum(prior, 1e-300))[None]
185         w = np.exp(logits-logits.max(1, keepdims=True)); w /= w.sum(1, keepdims=True)
186         return w
187
188
189     class EnhancedMarkovDP:
190         """Origin-conditioned VAR + empirical innovations, convex inventory chords.
191
192         External states use scenario support with nonnegative kernel interpolation.
193         This is an approximate finite-support kernel, not a true-process lower bound.
194         Current price is integrated before choosing the physical action.
195         """
196         def __init__(self, bundle, q, asof, config, tail_price, terminal=None, tail_cuts=None):
197             net, prices, states, weights = (bundle[k] for k in ('net', 'prices', 'states', 'weights'))
198             S, T = net.shape; D = states.shape[2]; self.grid = np.linspace(EMIN, EMAX, config.grid)
199             self.support = states; self.weights = weights; self.variable = D == 4
200             self.future = np.empty((T, S, config.grid)); self.price = np.empty((T, S))
201             self.models = []; self.alpha = config.alpha; self.forecast_price = bundle['forecast']['price']
202             hx = bundle['history_states']; rn, rp = bundle['history_net_error'], bundle['history_price_error']
203             X = hx[:, :-1].reshape(-1, D)
204             Y = np.stack((rn[:, 1:], rp[:, :-1]), axis=2).reshape(-1, 2) if self.variable else rn[:, 1:].reshape(-1,
205             ↪ 1)
206             phases = ((bundle['origins'][:, None]+np.arange(T-1)) % 144//36).ravel()
207             scales = np.std(X, axis=0)
208             # Each error and corresponding EMA share a scale to preserve the EMA algebra.
209             scales[2:] = max(float(np.std(rn)), 1e-8)
210             if self.variable: scales[2:] = max(float(np.std(rp)), 1e-8)
211             self.scales = scales
212             for phase in range(4):

```

```

212     ids = np.flatnonzero(phases == phase)
213     if not len(ids): ids = np.arange(len(X))
214     design = np.c_[np.ones(len(ids)), X[ids]/scales]
215     regularizer = np.diag(np.r_[0., np.full(D, config.ridge*len(ids))])
216     coefficients = np.linalg.solve(design.T@design+regularizer, design.T@Y[ids])
217     residuals = Y[ids]-design@coefficients
218     # Deterministic farthest-first representatives, masses assigned by nearest
219     # empirical innovation. Actual historical residuals, never Gaussian draws.
220     residual_scale = np.maximum(residuals.std(0), 1e-8)
221     standardized = residuals/residual_scale
222     chosen = [int(np.argmin(np.sum(standardized**2, axis=1)))]
223     ↪ np.sum((standardized-standardized[chosen[0]])**2, axis=1)
224     for _ in range(min(config.innovation_count, len(ids))-1):
225         ix = int(np.argmax(distance))
226         if distance[ix] < 1e-14: break
227         chosen.append(ix); distance = np.minimum(distance, np.sum((standardized-standardized[ix])**2,
228         ↪ axis=1))
229     labels = np.argmax(cdist(standardized, standardized[chosen]), axis=1)
230     mass = np.bincount(labels, minlength=len(chosen))/len(labels)
231     selected = residuals[chosen]; mean_error = mass@selected
232     # Keep unshifted empirical innovations; report reduced-measure mean error.
233     a = coefficients[1:].T/scales[None]; b = coefficients[0]
234     A = np.empty((D, D)); intercept = np.empty(D)
235     A[0] = a[0]; A[1] = config.alpha*a[0]; A[1, 1] += 1-config.alpha
236     intercept[:2] = b[0], config.alpha*b[0]
237     if self.variable:
238         A[2] = a[1]; A[3] = config.alpha*a[1]; A[3, 3] += 1-config.alpha
239         intercept[2:] = b[1], config.alpha*b[1]
240     self.models.append({'A': A, 'b': intercept, 'innovations': selected, 'mass': mass,
241     ↪ 'rows': len(ids), 'reduced_innovation_mean': mean_error})
242     v = np.tile(terminal_values(self.grid, tail_price, terminal, tail_cuts), (S, 1))
243     for j in range(T-1, -1, -1):
244         phase = (asof+j) % 144//36
245         nxt, mean_price = self.transition(states[:, j], phase, self.forecast_price[j])
246         self.price[j] = mean_price
247         if j == T-1:
248             future = np.tile(v[0], (S, 1))
249         else:
250             model = self.models[phase]; kernels = nonnegative_kernel(states[:, j+1], nxt.reshape(-1, D),
251             ↪ scales, weights)
252             P = np.einsum('sik,i->sk', kernels.reshape(S, -1, S), model['mass'])
253             future = P@v
254         self.future[j] = future
255         # Collocate the SAME continuous kernel used by action(). A Gaussian
256         # kernel is not cardinal: even at a support point it mixes neighbors.
257         # Applying it only online would define a different greedy policy.
258         support_kernel = nonnegative_kernel(states[:, j], states[:, j], scales, weights)
259         collocated_future = support_kernel@future
260         collocated_price = support_kernel@mean_price
261         v = np.array([inf_convolution(self.grid, collocated_future[s], net[s, j]-q[j], collocated_price[s])
262         ↪ for s in range(S)])
263     self.initial_values = v
264     self.asof = asof
265
266     def transition(self, x, phase, baseline_price):
267         x = np.atleast_2d(x); model = self.models[phase]; innovation = model['innovations']
268         predicted = x@model['A'].T+model['b']
269         next_state = np.repeat(predicted[:, None], len(innovation), axis=1)
270         next_state[:, :, 0] += innovation[:, 0]; next_state[:, :, 1] += self.alpha*innovation[:, 0]
271         if self.variable:
272             error = predicted[:, None, 2]+innovation[None, :, 1]
273             price = np.maximum(.001, baseline_price+error)
274             error = price-baseline_price
275             next_state[:, :, 2] = error
276             next_state[:, :, 3] = (1-self.alpha)*x[:, None, 3]+self.alpha*error
277             mean_price = price@model['mass']
278         else:
279             mean_price = np.full(len(x), baseline_price)
280         return next_state, mean_price
281
282     def action(self, j, E, net, q, state):
283         w = nonnegative_kernel(self.support[:, j], state, self.scales, self.weights)[0]
284         # The same nonnegative interpolation defines the declared approximate kernel
285         # and its continuation value; coefficients never depend on inventory.

```



```

282     future = w@self.future[j]
283     price = float(w@self.price[j])
284     return minimize_action(self.grid, future, E, net-q, price)
285
286 def diagnostics(self):
287     return {'model': 'origin-conditioned empirical VAR with convex inventory interpolation',
288           'scale': self.scales.tolist(), 'phases': [
289               {k: v.tolist() if isinstance(v, np.ndarray) else v for k, v in model.items()}
290               for model in self.models]}

```

F.6 年度逐段执行

```
1 """Incremental chronological replay with the frozen original meter/executor."""
2 from pathlib import Path
3 from dataclasses import asdict
4 import argparse
5 import hashlib
6 import json
7 import time
8 import numpy as np
9 from control import ETA, execute, prune_cuts
10 from dispatch import settlement
11 from nextgen_scenarios import Config, ScenarioFactory, weighted_quantile
12 from nextgen_control import affine_plan, WeightedMarkovDP, EnhancedMarkovDP
13
14 ROOT = Path(__file__).resolve().parents[1]
15 KINDS = {'q2': (False, False), 'q3': (True, False), 'q4_2': (False, True), 'q4_3': (True, True)}
16
17
18 def dump(path, obj):
19     path = Path(path); path.parent.mkdir(parents=True, exist_ok=True)
20     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
21
22
23 def simulate(data, selection, days, kind, config, initial=6000., progress=None):
24     source_hashes = {p.name: hashlib.sha256(p.read_bytes()).hexdigest() for p in
25 ↪ sorted((ROOT/'src').glob('*.py'))}
26     official, variable = KINDS[kind]; days = list(days); count = len(days)
27     arrays = {k: np.zeros((count, 144)) for k in ['q', 'r', 'c', 'd', 'emergency', 'spill', 'price',
28 ↪ 'projection']}
29     arrays['state'] = np.zeros((count, 145)); arrays['releases'] = np.full((count, 4, 144), np.nan)
30     factory = ScenarioFactory(data, selection, official, variable, config.alpha,
31 ↪ config.fusion_weight, config.official_correction)
32     started = time.perf_counter(); E = float(initial); evaluation_end = (days[-1]+1)*144
33     decisions = []; daily = []; lp_violation = objective_gap = 0.; tails = {}
34     planning_seconds = feedback_seconds = scenario_seconds = 0.
35     for i, day in enumerate(days):
36         arrays['state'][i, 0] = E
37         actual = (data['load'][day]-data['pv'][day])/6
38         scoring_price = data['price'][day] if variable else data['day_price']
39         arrays['price'][i] = scoring_price
40         for t in range(144):
41             now = day*144+t
42             if t % 36 == 0:
43                 end = min(now+6*config.horizon_hours if config.horizon_mode == 'fixed'
44 ↪ else (day+config.horizon_hours//24)*144, evaluation_end)
45                 stage_started = time.perf_counter(); bundle = factory.build(now, end, config)
46                 scenario_seconds += time.perf_counter()-stage_started
47                 net, prices, weights = (bundle[k] for k in ('net', 'prices', 'weights'))
48                 base = None if t == 0 else np.r_[arrays['q'][i, t:], np.zeros(end-(day+1)*144)]
49                 terminal = config.final if end == evaluation_end else None
50                 lam = 0.; cuts = None; cutoff = tail_days = None
51                 if end < evaluation_end:
52                     if config.tail == 'linear':
53                         last_prices = prices[:, -min(144, end-now):]
54                         lam = config.tail_scale*float(weighted_quantile(last_prices.ravel(), [.25],
55 ↪ np.repeat(weights, last_prices.shape[1]))[0])/ETA
56                     elif config.tail == 'sddp':
57                         if end % 144: raise ValueError('Existing SDDP tail is valid only at midnight')
58                         cutoff = 14 if day < 31 else day-int(str(data['dates'][day])[8:10])+1
59                         tail_days = min(3, (evaluation_end-end)//144)
60                         suffix = '' if tail_days == 3 else '_D'+str(tail_days)
61                         path = ROOT.parent/'C 题 _ 下一代基线复现'/f'artifacts/tails/{kind}_{cutoff}-{suffix}.json'
62                         if path not in tails: tails[path] = json.loads(path.read_text())
63                         assert tails[path]['configuration']['cutoff_day_exclusive']*144 <= now
64                         cuts = prune_cuts(tails[path]['root_cuts'])
65                     elif config.tail != 'zero': raise ValueError(config.tail)
66                 plan = affine_plan(bundle, now, E, config, base, official, lam, terminal, cuts)
67                 planning_seconds += plan['runtime_seconds']
68                 lp_violation = max(lp_violation, plan['lp_violation']); objective_gap = max(objective_gap,
69 ↪ plan['objective_gap'])
70             if t == 0:
71                 arrays['q'][i] = np.maximum(plan['q'][:144], 0); arrays['r'][i] = arrays['q'][i]
72                 arrays['releases'][i, 0] = arrays['q'][i]
```

```

70         elif official:
71             arrays['r'][i, t:] = np.maximum(plan['r'][:144-t], 0)
72             arrays['releases'][i, t//36, t:] = arrays['r'][i, t:]
73             effective = plan['r'].copy(); locked = 36 if official else 144-t
74             effective[:locked] = arrays['r'][i, t:t+locked]
75             stage_started = time.perf_counter()
76             if config.lower == 'M0':
77                 dp = WeightedMarkovDP(net, prices, effective, weights, lam, config.grid, config.bins,
78                                     ↪ terminal, cuts)
79             elif config.lower == 'M1':
80                 dp = EnhancedMarkovDP(bundle, effective, now, config, lam, terminal, cuts)
81             else: raise ValueError(config.lower)
82             feedback_seconds += time.perf_counter()-stage_started
83             active = t; zn, ep, zp = bundle['seed'][1:]
84             record = {'**bundle': bundle['meta'], 'date': str(data['dates'][day]), 'release': t//36,
85                     'inventory': E, 'tail_price': lam, 'tail_model_cutoff': cutoff,
86                     'tail_model_days': tail_days, 'tail_cut_count': len(cuts) if cuts else 0,
87                     'contract_information_cutoff': now, 'current_delivery_observation_in_contract': False,
88                     'gain_active_count': plan['gain_active_count'], 'gain_bound_hit_count':
89                     ↪ plan['gain_bound_hit_count'],
90                     'gain_max_abs': plan['gain_max_abs'], 'lp_violation': plan['lp_violation'],
91                     'objective_recompute_gap': plan['objective_gap'], 'solver_seconds':
92                     ↪ plan['runtime_seconds'],
93                     'partial_contract_completion': plan['future_partial_contract_completion'],
94                     'contract_information_cutoffs': plan['contract_information_cutoffs']}
95             if config.lower == 'M1': record['enhanced_markov'] = dp.diagnostics()
96             decisions.append(record)
97             j = t-active
98             error = actual[t]-bundle['forecast']['net'][j]; zn = (1-config.alpha)*zn+config.alpha*error
99             state = np.array([error, zn, ep, zp]) if variable else np.array([error, zn])
100             target = dp.action(j, E, actual[t], arrays['r'][i, t], state)
101             E, c, discharge, emergency, spill, projection = execute(E, target, arrays['r'][i, t], actual[t],
102             ↪ evaluation_end-now-1, final=config.final)
103             for name, value in [(('c', c), ('d', discharge), ('emergency', emergency), ('spill', spill),
104             ↪ ('projection', projection))]:
105                 arrays[name][i, t] = value
106             arrays['state'][i, t+1] = E
107             # Price is disclosed only after the battery action has been committed.
108             ep = data['price'][day, t]-bundle['forecast']['price'][j]
109             zp = (1-config.alpha)*zp+config.alpha*ep
110             cost = settlement(arrays['q'][i], arrays['r'][i], arrays['emergency'][i], scoring_price)
111             daily.append({'date': str(data['dates'][day]), **cost,
112                     'emergency_energy': float(arrays['emergency'][i].sum()),
113                     'spill_energy': float(arrays['spill'][i].sum()), 'ending_inventory': E,
114                     'projection_count': int(np.sum(arrays['projection'][i] > 1e-6))})
115             if progress and ((i+1) % 7 == 0 or i+1 == count):
116                 dump(progress, {'kind': kind, 'configuration_id': config.identity(), 'completed_days': i+1,
117                     'requested_days': count, 'last_date': str(data['dates'][day]),
118                     'elapsed_seconds': time.perf_counter()-started, 'status': 'running'})
119             balance = arrays['r']+arrays['emergency']+arrays['d']-arrays['c']-arrays['spill']-(data['load'][days]-data['p']
120             ↪ pv)[days])/6
121             validation = {'balance_max_abs': float(np.abs(balance).max()),
122                     'soc_recurrence_max_abs': float(np.abs(np.diff(arrays['state'],
123                     ↪ axis=1)-ETA*arrays['c']+arrays['d']/ETA).max()),
124                     'midnight_jump_max_abs': float(np.abs(arrays['state'][1:, 0]-arrays['state'][:-1, -1]).max(initial=0)),
125                     'soc_min': float(arrays['state'].min()), 'soc_max': float(arrays['state'].max()),
126                     'power_max_kw': float(6*max(arrays['c'].max(), arrays['d'].max())),
127                     'simultaneous_max': float(np.minimum(arrays['c'], arrays['d']).max()),
128                     'initial_inventory': initial, 'final_inventory': E, 'lp_violation_max': lp_violation,
129                     'lp_objective_recompute_gap': objective_gap}
130             assert validation['balance_max_abs'] < 1e-5 and validation['soc_recurrence_max_abs'] < 1e-5
131             assert validation['midnight_jump_max_abs'] < 1e-5 and validation['soc_min'] >= 1200-1e-5 and
132             ↪ validation['soc_max'] <= 10800+1e-5
133             assert config.final is None or abs(E-config.final) < 1e-5
134             arrays['days'] = np.array(days); arrays['dates'] = data['dates'][days]
135             keys = ['planned_cost', 'increase_cost', 'reduction_net_cost', 'emergency_cost', 'total_cost',
136             ↪ 'emergency_energy', 'spill_energy', 'projection_count']
137             metrics = {'kind': kind, 'candidate': config.identity(),
138                     'configuration': {'**asdict(config), 'initial': initial, 'closed_daily': False, 'start_day':
139                     ↪ days[0], 'end_day': days[-1]},
140                     'totals': {k: float(sum(row[k] for row in daily)) for k in keys},
141                     'validation': validation, 'daily': daily, 'decisions': decisions,
142                     'runtime_seconds': time.perf_counter()-started,

```

```

133         'timing': {'planning_seconds': planning_seconds, 'feedback_build_seconds': feedback_seconds,
↪      'scenario_seconds': scenario_seconds},
134         'source_hashes': source_hashes}
135     assert source_hashes == {p.name: hashlib.sha256(p.read_bytes()).hexdigest() for p in
↪      sorted((ROOT/'src').glob('*.py'))}, 'Source changed during replay; do not certify this run'
136     return arrays, metrics
137
138
139 def run_case(kind, configuration, start, stop, output):
140     config = Config(**configuration); data = dict(np.load(ROOT/'artifacts/data.npz'))
141     selection = json.loads((ROOT/'artifacts/forecast-selection.json').read_text())
142     output = Path(output); output.parent.mkdir(parents=True, exist_ok=True)
143     arrays, metrics = simulate(data, selection, range(start, stop), kind, config,
↪      progress=output.with_suffix('.progress.json'))
144     np.savez_compressed(output.with_suffix('.npz'), **arrays); dump(output.with_suffix('.json'), metrics)
145     dump(output.with_suffix('.progress.json'), {'status': 'complete', 'completed_days': stop-start})
146     return {'kind': kind, 'configuration': configuration, 'output': str(output), 'cost':
↪      metrics['totals']['total_cost'], 'runtime_seconds': metrics['runtime_seconds']}
147
148
149 if __name__ == '__main__':
150     p = argparse.ArgumentParser(); p.add_argument('--kind', default='q2'); p.add_argument('--config',
↪      default='{ }')
151     p.add_argument('--start', type=int, default=24); p.add_argument('--stop', type=int, default=31)
152     p.add_argument('--output', default=str(ROOT/'artifacts/development/single'))
153     args = p.parse_args(); print(json.dumps(run_case(args.kind, json.loads(args.config), args.start, args.stop,
↪      args.output), ensure_ascii=False))

```

F.7 辅助继续值模块

```

1  """Finite-horizon, risk-neutral SDDP for general convex linear stage models.
2
3  The engine only requires solve(state, noise, next_cuts) -> objective, outgoing
4  state, realized stage cost, input-state dual. BatteryStage is one application.
5  Stagewise-independent noises are observed at stage entry. Here-and-now
6  contracts have their own preceding deterministic stages.
7  """
8  from pathlib import Path
9  from dataclasses import dataclass
10 import sys,json,time,hashlib,argparse
11 import numpy as np
12 from scipy.sparse import coo_matrix,vstack,csr_matrix
13 sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'vendor'))
14 import highspy
15
16 @dataclass
17 class StageResult:
18     objective:float
19     state:np.ndarray
20     stage_cost:float
21     gradient:np.ndarray
22
23 class LinearStage:
24     """General LP stage with explicit incoming-state fixing equalities.
25
26     Variables start with x_in, x_out, then stage controls, final theta.
27     The user supplies base matrix/bounds/cost for each finite noise outcome.
28     Costs must exclude theta, which the engine adds with coefficient one.
29     """
30     def __init__(self,dimension,outcomes):
31         self.dimension=dimension;self.outcomes=outcomes
32         self.proBABILITIES=np.array([o['probability'] for o in outcomes])
33         assert np.isclose(self.proBABILITIES.sum(),1)
34     def solve(self,state,noise,next_cuts):
35         o=self.outcomes[noise];D=self.dimension;N=len(o['cost']);theta=N-1
36         fix=coo_matrix((np.ones(D),(np.arange(D),np.arange(D))),shape=(D,N)).tocsr()
37         mats=[fix,o['matrix']];lower=[state,o['row_lower']];upper=[state,o['row_upper']]
38         if next_cuts:
39             coeff=np.array([c[1] for c in next_cuts]);rows,cols=np.nonzero(coeff)
40             mat=coo_matrix((np.r_[coeff[rows,cols],-np.ones(len(next_cuts))],(np.r_[rows,np.arange(len(next_cuts)
41             ↪ ))],np.r_[D+cols,np.full(len(next_cuts),theta)]),shape=(len(next_cuts),N)).tocsr()
42             mats.append(mat);lower.append(np.full(len(next_cuts),-np.inf));upper.append(-np.array([c[0] for c in
43             ↪ next_cuts]))
44         A=.vstack(mats,format='csr');lo=np.concatenate(lower);hi=np.concatenate(upper)
45         p=highspy.HighsLp();p.num_col_=N;p.num_row_=len(lo);p.col_cost_=o['cost'].copy();p.col_cost_[theta]=1.
46         p.col_lower_=o['lower'];p.col_upper_=o['upper'];p.row_lower_=lo;p.row_upper_=hi
47         p.a_matrix_.format=highspy.MatrixFormat.kRowwise;p.a_matrix_.start_=A.indptr;p.a_matrix_.index_=A.indic
48         ↪ es;p.a_matrix_.value_=A.data
49         h=highspy.Highs();h.setOptionValue('output_flag',False);h.setOptionValue('threads',1);h.passModel(p);h.r
50         ↪ un()
51         if h.getModelStatus()!=highspy.HighsModelStatus.kOptimal:raise
52         ↪ RuntimeError(h.modelStatusToString(h.getModelStatus()))
53         sol=h.getSolution();x=np.array(sol.col_value)
54         return StageResult(h.getObjectiveValue(),x[D:2*D],float(o['cost']@x),np.array(sol.row_dual[:D]))
55
56 class SDDP:
57     def __init__(self,stages,seed=20260911):
58         self.stages=stages;self.dimension=stages[0].dimension;self.rng=np.random.default_rng(seed);self.seed=seed
59         self.cuts=[[] for _ in range(len(stages)+1)];self.trace=[]
60     def forward(self,initial):
61         state=np.array(initial,dtype=float);inputs=[];cost=0.
62         for t,stage in enumerate(self.stages):
63             inputs.append(state.copy());k=int(self.rng.choice(len(stage.outcomes),p=stage.proBABILITIES))
64             sol=stage.solve(state,k,self.cuts[t+1]);state=sol.state;cost+=sol.stage_cost
65         return inputs,cost
66     def backward(self,inputs):
67         for t in range(len(self.stages)-1,-1,-1):
68             x=inputs[t];stage=self.stages[t]
69             vals=[stage.solve(x,k,self.cuts[t+1]) for k in range(len(stage.outcomes))]
70             value=sum(p*v.objective for p,v in zip(stage.proBABILITIES,vals))
71             beta=sum(p*v.gradient for p,v in zip(stage.proBABILITIES,vals));alpha=float(value-beta*x)
72             self.cuts[t].append((alpha,beta))

```

```

68     def bound(self, initial):
69         stage=self.stages[0]
70         return float(sum(p*stage.solve(np.array(initial),k,self.cuts[1]).objective for k,p in
        ↪ enumerate(stage.probabilities)))
71     def train(self, initial, iterations=50, exploration=None):
72         for it in range(iterations):
73             x=np.array(initial, dtype=float)
74             if exploration is not None: x=exploration(it, x)
75             path, cost=self.forward(x); self.backward(path)
76             if it==0 or (it+1)%10==0 or it==iterations-1: self.trace.append({'iteration': it+1, 'root_lower_bound': }
        ↪ self.bound(initial), 'forward_sample_cost': cost})
77         return self
78     def evaluate(self, initial, count=128, seed=9182):
79         old=self.rng; self.rng=np.random.default_rng(seed)
80         costs=np.array([self.forward(initial)[1] for _ in range(count)]); self.rng=old
81         se=float(costs.std(ddof=1)/np.sqrt(count))
82         return {'samples': count, 'seed': seed, 'mean_policy_cost': float(costs.mean()), 'standard_error': se,
83                 'normal_approximation_95_interval': [float(costs.mean()-1.96*se), float(costs.mean()+1.96*se)],
84                 'root_lower_bound': self.bound(initial), 'note': 'Monte Carlo estimate, not a deterministic upper
        ↪ bound.'}
85
86     def battery_stage(K, kind, hour, net_values=None, prices=None, probabilities=None):
87         D=1+2*K; N=2*D+3; em=2*D; bill=em+1; theta=bill+1
88         # Contract states contain all K deliveries; delivered entries are set to zero.
89         rows=[]; cols=[]; vs=[]; rl=[]; ru=[]
90         def row(items, lo, hi):
91             r=len(rl)
92             for c, v in items.items(): rows.append(r); cols.append(c); vs.append(v)
93             rl.append(lo); ru.append(hi)
94         def copy(i): row({D+i: 1, i: -1}, 0, 0)
95         q=lambda h: 1+h
96         r=lambda h: 1+K+h
97         if kind=='sign':
98             copy(0)
99             for h in range(K): row({D+r(h): 1, D+q(h): -1}, 0, 0)
100         elif kind=='revise':
101             copy(0)
102             for h in range(K):
103                 copy(q(h))
104                 if h<hour: copy(r(h))
105         else:
106             dt=24/K; max_bus=5000*dt
107             row({D: 1, 0: -1}, -max_bus/.9, .9*max_bus)
108             for h in range(K):
109                 if h==hour:
110                     row({D+q(h): 1}, 0, 0); row({D+r(h): 1}, 0, 0)
111                 else: copy(q(h)); copy(r(h))
112             row({bill: 1, r(hour): -1.5, q(hour): .5}, 0, np.inf)
113             row({bill: 1, r(hour): -.5, q(hour): -.5}, 0, np.inf)
114             row({em: 1, r(hour): 1, D: -.9, 0: .9}, 0, np.inf)
115             row({em: 1, r(hour): 1, D: -1/.9, 0: 1/.9}, 0, np.inf)
116         matrix=coo_matrix((vs, (rows, cols)), shape=(len(rl), N)).tocsr()
117         lower=np.zeros(N); upper=np.full(N, np.inf); lower[0]=lower[D]=1200; upper[0]=upper[D]=10800
118         outcomes=[]
119         if kind!='deliver': net_values=[0]; prices=[0]; probabilities=[1]
120         for net, p, prob in zip(net_values, prices, probabilities):
121             cost=np.zeros(N); cost[em]=5*p; cost[bill]=p
122             lowers=np.array(rl, dtype=float)
123             if kind=='deliver': lowers[-2:]=net
124             outcomes.append({'probability': float(prob), 'matrix': matrix, 'row_lower': lowers, 'row_upper': np.array(ru, dt }
        ↪ ype=float),
125                             'lower': lower, 'upper': upper, 'cost': cost})
126         return LinearStage(D, outcomes)
127
128     def fit_battery_model(data, cutoff, variable=False, adjust=False, K=12, days=3, window=28):
129         """Coarse auxiliary future model, fitted only to complete past days.
130
131         Per-stage net bins and conditional price means form a declared independent
132         finite innovation model. This model is a terminal-value proxy, not a lower
133         bound for the richer forecast-conditioned contest controller.
134         """
135         assert 144%K==0 and K%4==0 and cutoff>=7
136         hist=np.arange(max(0, cutoff-window), cutoff); width=144//K
137         net=((data['load'][hist]-data['pv'][hist])/6).reshape(len(hist), K, width).sum(2)

```

```

138     p=(data['price'][hist] if variable else
↪     np.repeat(data['day_price'][None,:],len(hist),axis=0)).reshape(len(hist),K,width).mean(2)
139     stages=[];emissions=[]
140     for k in range(K):
141         edges=np.quantile(net[:,k],[1/3,2/3]);labels=(net[:,k]>edges[0]).astype(int)+(net[:,k]>edges[1]).astype(int)
↪         int)
142         vals=[];prices=[];weights=[]
143         for b in range(3):
144             ids=np.flatnonzero(labels==b)
145             if len(ids):vals.append(float(net[ids,k].mean()));prices.append(float(p[ids,k].mean()));weights.append(
↪             nd(len(ids)/len(hist))
146         emissions.append((vals,prices,weights))
147     for d in range(days):
148         stages.append(battery_stage(K,'sign',0))
149         for k in range(K):
150             if adjust and k and k%(K//4)==0:stages.append(battery_stage(K,'revise',k))
151             stages.append(battery_stage(K,'deliver',k,*emissions[k]))
152     return
↪     stages,{'cutoff_day_exclusive':cutoff,'history_days':hist.tolist(),'steps_per_day':K,'horizon_days':days,
153             'variable_price':variable,'adjustments':adjust,'noise_model':'Stagewise-independent net bins; conditional
↪             expected prices; hourly aggregation is an auxiliary-model approximation.'}
154
155 def train_tail(data,cutoff,variable=False,adjust=False,K=12,days=3,iterations=50,seed=20260911):
156     start=time.perf_counter();stages,meta=fit_battery_model(data,cutoff,variable,adjust,K,days)
157     model=SDDP(stages,seed);initial=np.r_[6000.,np.zeros(2*K)]
158     def exploration(i,x):x[0]=np.linspace(1200,10800,9)[i%9];return x
159     model.train(initial,iterations,exploration)
160     ev=model.evaluate(initial,count=64)
161     # Root signing resets contracts: its value depends on incoming inventory only.
162     assert max(np.max(np.abs(b[1:])) for a,b in model.cuts[0])<1e-8
163     rootcuts=[[a,float(b[0])] for a,b in model.cuts[0]]
164     return {'configuration':meta,'seed':seed,'iterations':iterations,'trace':model.trace,'evaluation':ev,
165           'root_cuts':rootcuts,'runtime_seconds':time.perf_counter()-start,
166           'scope':'Valid lower cuts for this fixed coarse auxiliary model; approximate terminal values for the
↪           contest MPC.'}
167
168 def main():
169     p=argparse.ArgumentParser();p.add_argument('--cutoff',type=int,default=31);p.add_argument('--iterations',type=
↪     int,default=50);p.add_argument('--steps',type=int,default=12);p.add_argument('--days',type=int,default=
↪     3);p.add_argument('--variable',action='store_true');p.add_argument('--adjust',action='store_true');p.add
↪     argument('--output',default='sddp-pilot.json');args=p.parse_args()
170     root=Path(__file__).resolve().parents[1];data=dict(np.load(root/'artifacts/data.npz'))
171     out=train_tail(data,args.cutoff,args.variable,args.adjust,args.steps,args.days,args.iterations)
172     out['execution']={'command':'python3 '+'
↪     '.join(sys.argv),'exit_code':0,'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
173     (root/'artifacts'/args.output).write_text(json.dumps(out,ensure_ascii=False,indent=2)+'\n');print(json.dumps(
↪     out['evaluation']),flush=True)
174
175 if __name__=='__main__':main()

```